

Retrieval Augmented Generation

Paul Hongsuck Seo
phseo@korea.ac.kr
Korea University



KOREA
UNIVERSITY



Multimodal Interactive
Intelligence Laboratory

LLMs and their Limitations

Language Modeling

- The task of predicting *the next word* in a document.
- This technique can be used to train language models.
- Language models can be applied to a wide range of tasks:
 - Text generation, text completion, question answering.

$$P(x_1, x_2, \dots, x_T) = P(x_1)P(x_2|x_1) \cdots P(x_T|x_1, \dots, x_{T-1}) = \prod_{t=1}^T P(x_t|x_{<t})$$

I _____

Web search engine / ...

I saw a cat|

I saw a cat on the chair
I saw a cat running after a dog
I saw a cat in my dream
I saw a cat book

Keyboard / mail agent / ...

I saw a catt

cat
car

Language Models are Unsupervised Multitask Learners

- GPT-2
 - Language models can perform multiple downstream tasks without task-specific finetuning.
 - As the model size gets bigger, the zero-shot capability also improves

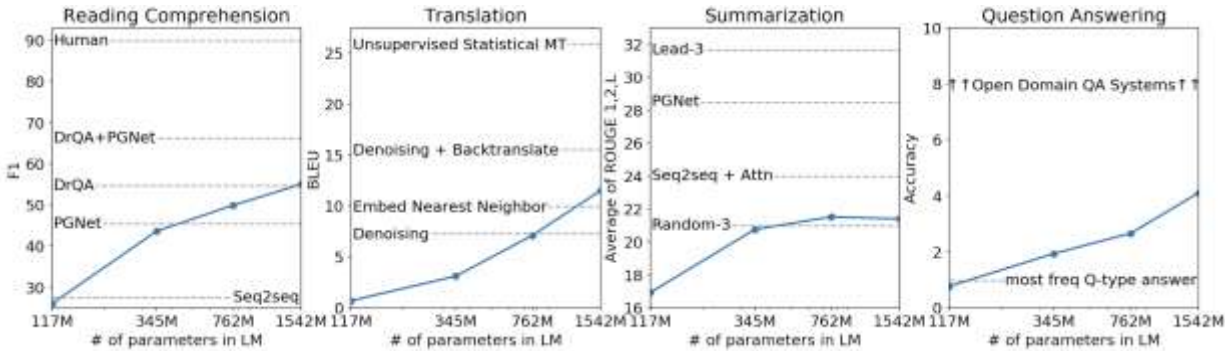


Figure 1. Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks. Reading Comprehension results are on CoQA (Reddy et al., 2018), translation on WMT-14 Fr-En (Artetxe et al., 2017), summarization on CNN and Daily Mail (See et al., 2017), and Question Answering on Natural Questions (Kwiatkowski et al., 2019). Section 3 contains detailed descriptions of each result.

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

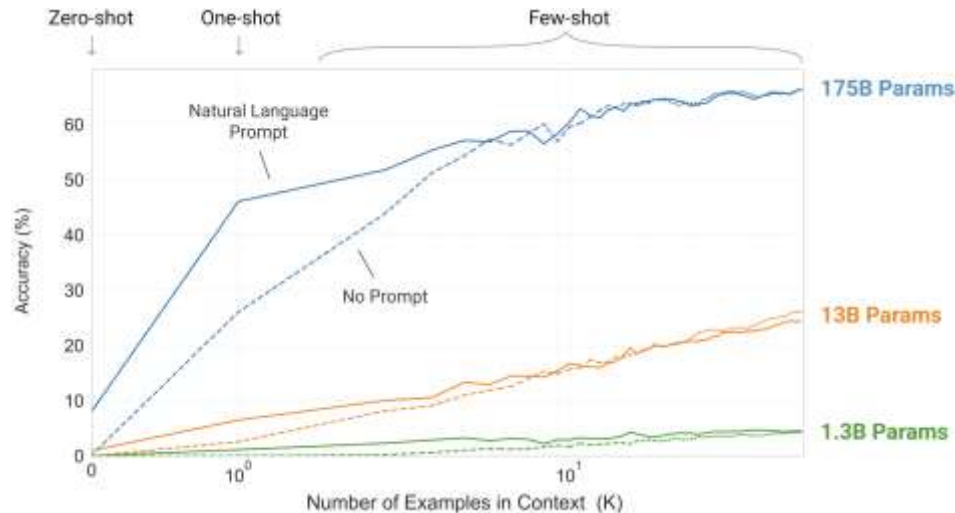
Table 3. Zero-shot results on many datasets. No training or fine-tuning was performed for any of these results. PTB and WikiText-2 results are from (Gong et al., 2018). CBT results are from (Bajgar et al., 2016). LAMBADA accuracy result is from (Hoang et al., 2018) and LAMBADA perplexity result is from (Grave et al., 2016). Other results are from (Dai et al., 2019).

	R-1	R-2	R-L	R-AVG
Bottom-Up Sum	41.22	18.68	38.34	32.75
Lede-3	40.38	17.66	36.62	31.55
Seq2Seq + Attn	31.33	11.81	28.83	23.99
GPT-2 TL; DR:	29.34	8.27	26.58	21.40
Random-3	28.78	8.63	25.52	20.98
GPT-2 no hint	21.58	4.03	19.47	15.03

Table 4. Summarization performance as measured by ROUGE F1 metrics on the CNN and Daily Mail dataset. Bottom-Up Sum is the SOTA model from (Gehrmann et al., 2018)

Language Models are Few-Shot Learners

- GPT-3
 - *In-context learning*: few shot examples given within the context improves the downstream predictions.



Brown et. al., Language Models are Few-Shot Learners, 2020

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 cheese => prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer example
3 cheese => prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer examples
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => prompt
```

Traditional fine-tuning (not used for GPT-3)

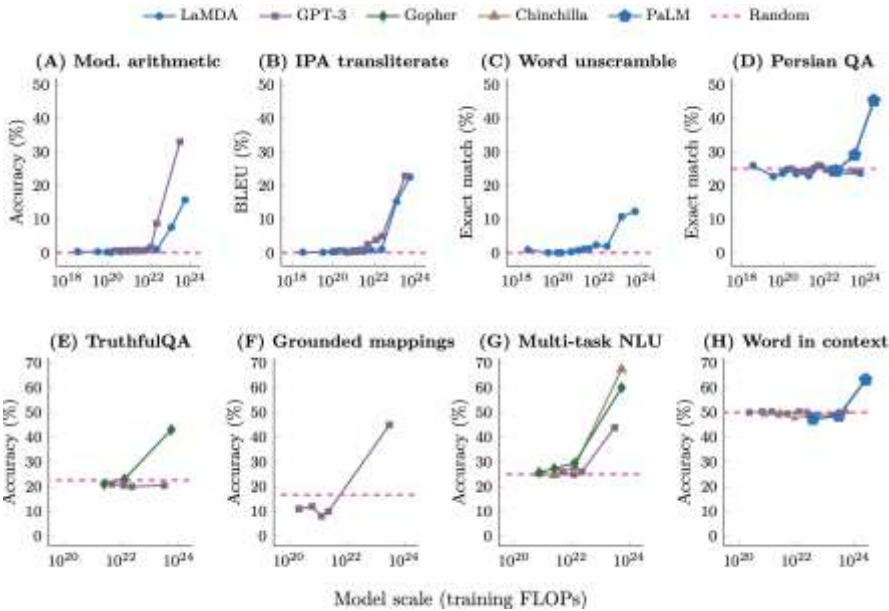
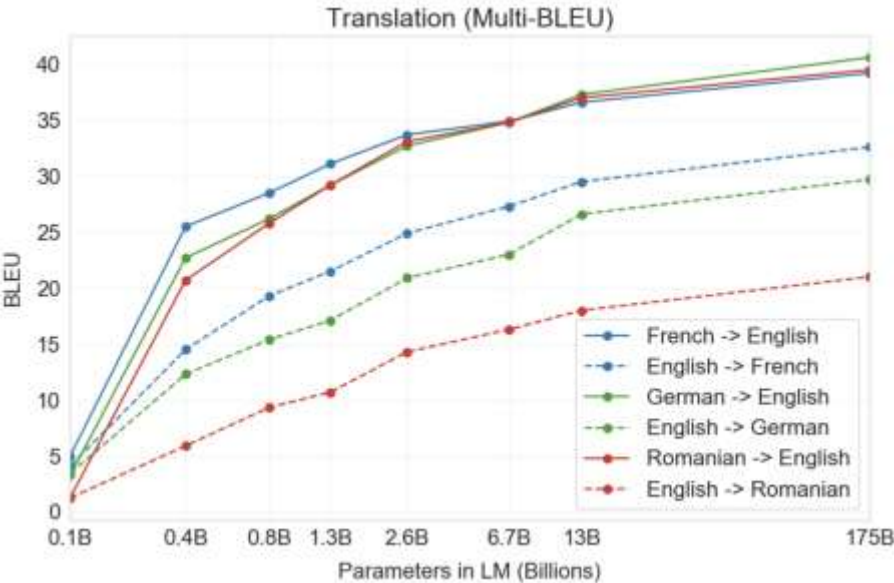
Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Language Models are Few-Shot Learners

- GPT-3
 - Model sizes matter.
 - New abilities are emergent as the model size becomes bigger.

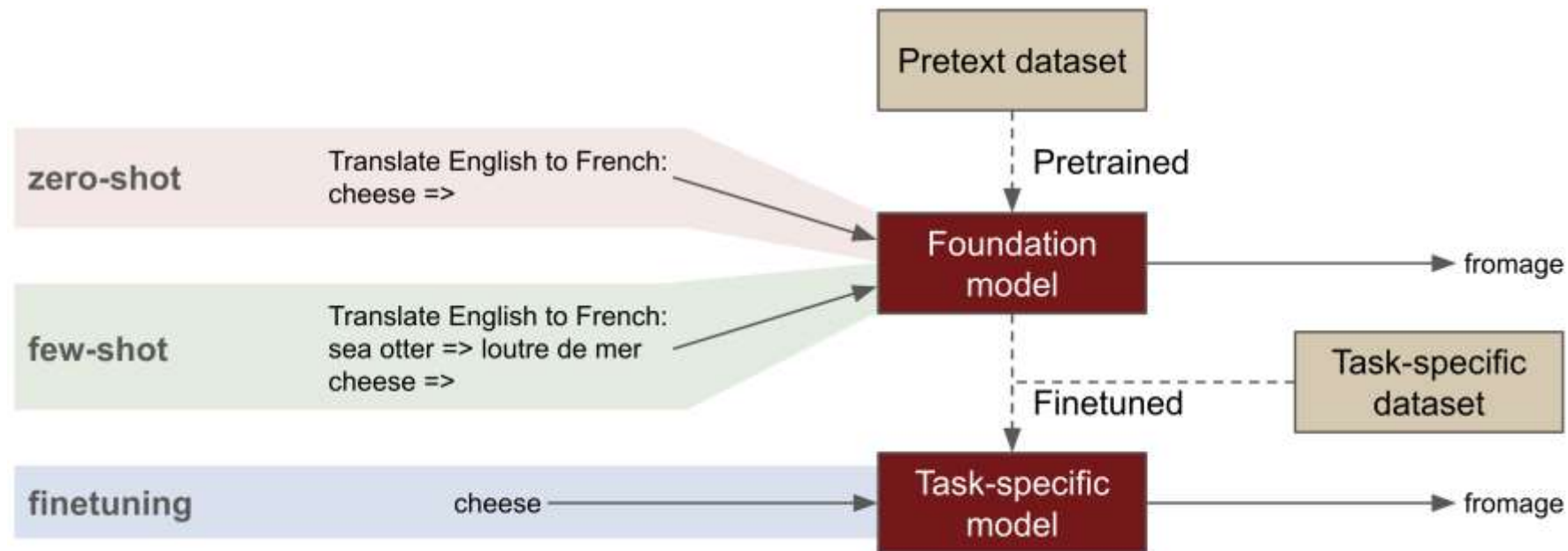


Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

Transferring Knowledge of FMs

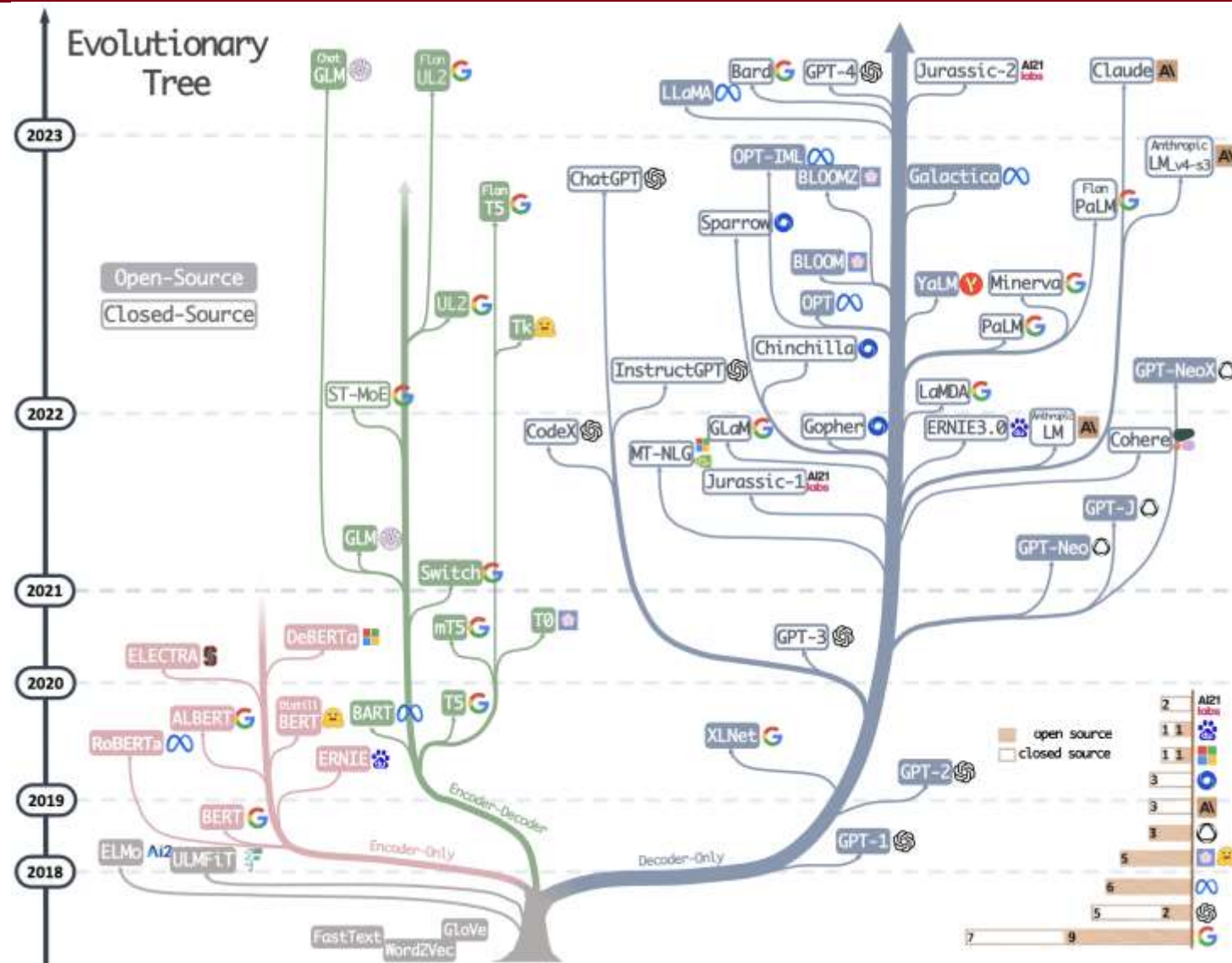
- **Data:** unannotated data, web-scale, multimodal
 - CLIP, Flamingo, SAM use (almost) annotated datasets.
- **Pretext objectives:** self-supervised learning
 - CLIP directly uses a retrieval objective.
- **Adaptation:** finetuning, prompt-engineering (zero/few-shot)



LLMs as Generalist AI Systems

- The Generalist Paradigm
 - A single LLM performs diverse tasks without task-specific training.
 - **Emergent Versatility:** Capability to perform coding, creative writing, and logical reasoning without retraining.
 - **Zero-Shot Learning:** The ability to solve novel problems purely through natural language instructions (prompting).

The Evolutionary Tree of Modern LLMs

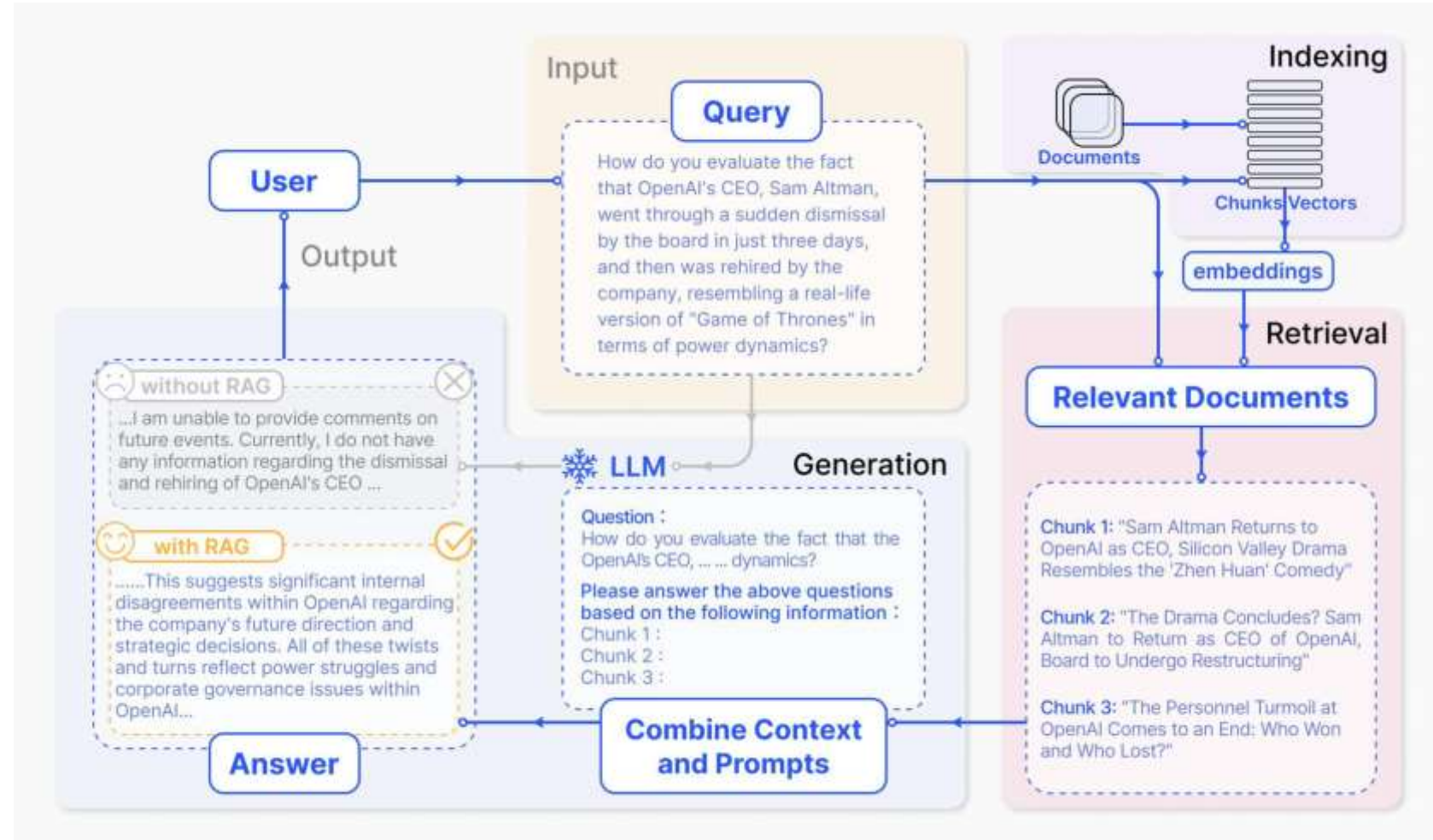


Challenges in LLMs

- Hallucination
 - Generating a response that is either factually incorrect, nonsensical, or disconnected from the input prompt.
- Outdated knowledge
 - Factual knowledge is naturally subject to change over time.
- Non-transparent, untraceable reasoning
 - LLMs operate as "black boxes," obscuring the logic behind their decisions.
- Lack of deep domain knowledge
 - LLMs trained on general-purpose datasets might lack the specific knowledge and context required for tasks in specialized fields.

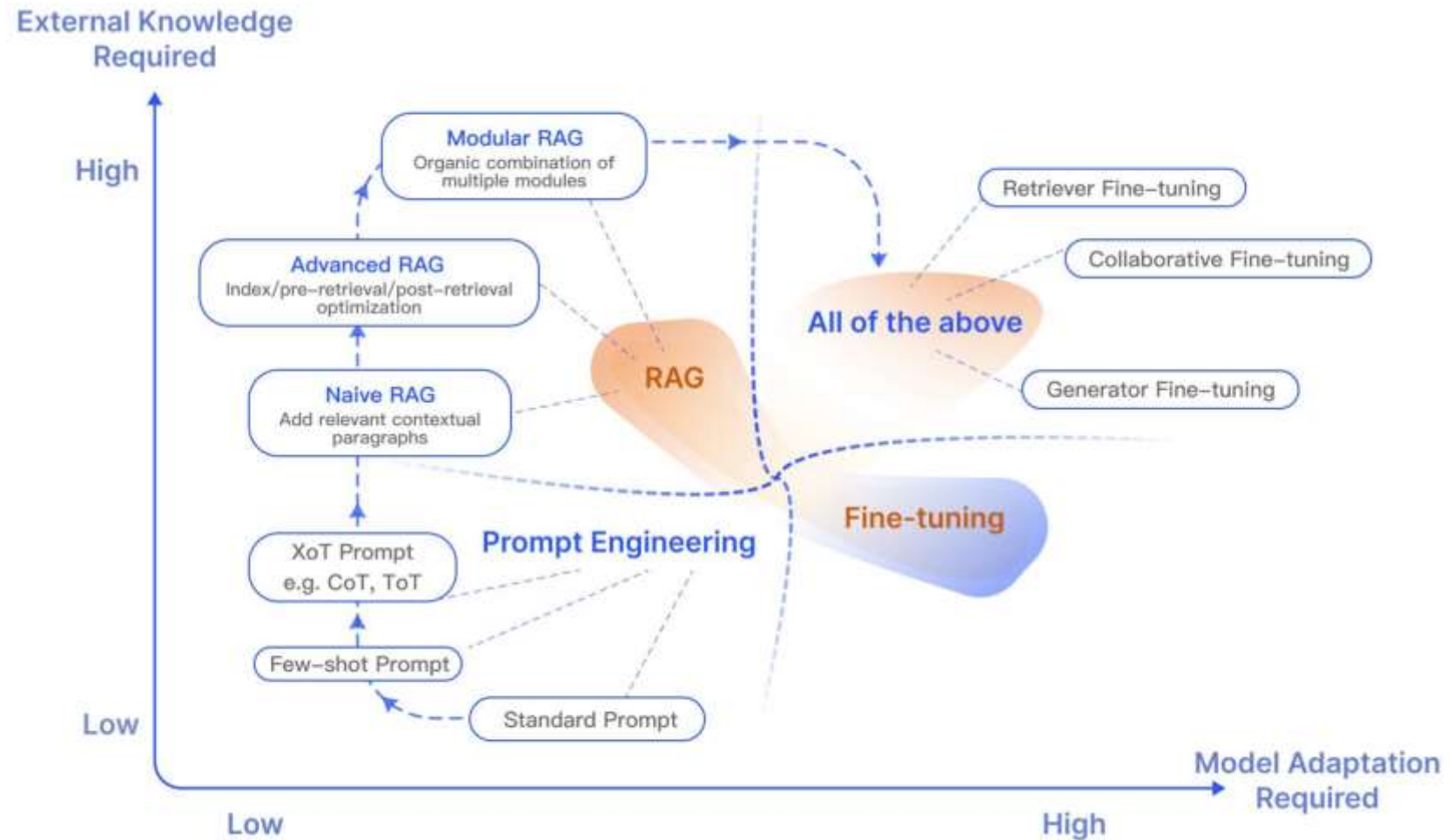
Retrieval Augmentation

- W/o RAG
 - Generate texts directly from the query.
 - *LLMs should contain everything.*
- W/ RAG
 - Retrieve relevant documents first.
 - Generate an output text given the retrieved documents.
 - *No need to retain entire knowledge within the model.*



Symbolic Knowledge or Parametric Knowledge

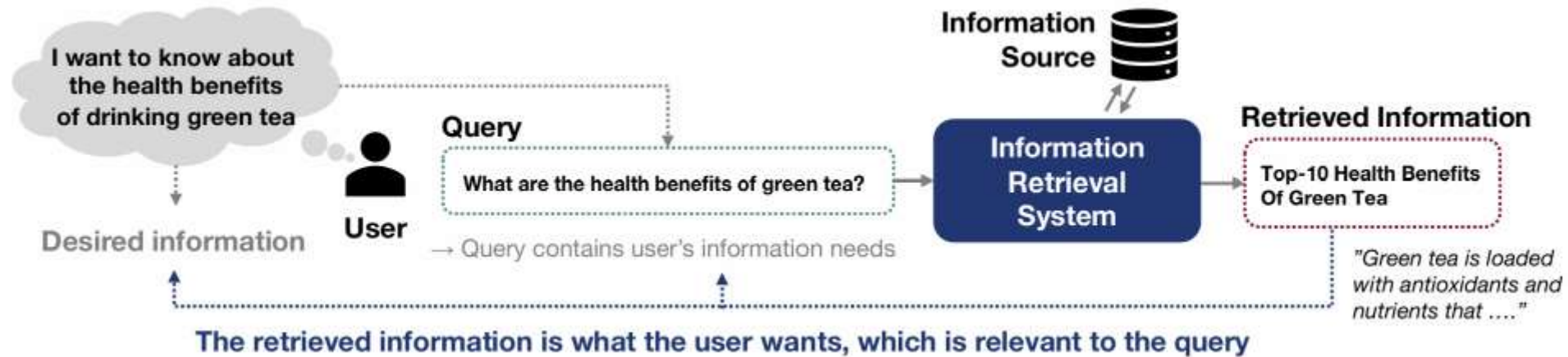
- Ways to optimize LLMs
 - Prompt engineering
 - Retrieval augmentation
 - Fine-tuning



Learning Retrievers

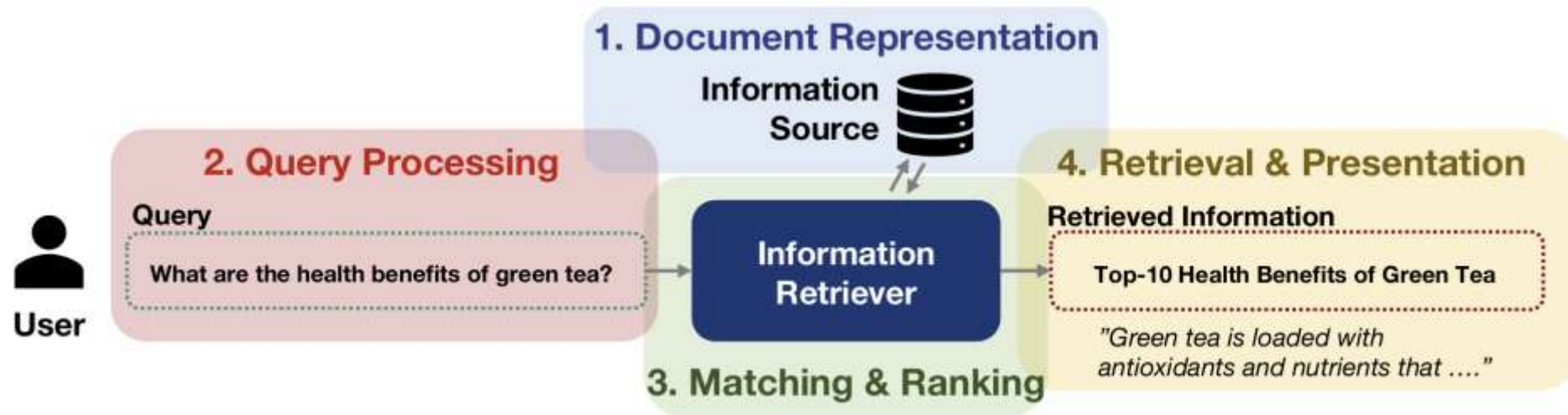
Information Retrieval

- Information Retrieval (IR) is the process of **obtaining relevant information** from a large repository, such as a database and the internet, based on a user's query.
- The primary goal is to provide the most relevant information to users in response to their search queries



Information Retrieval

- Key Components of Information Retrieval
 1. Document representation
 2. Query processing
 3. Matching and ranking
 4. Retrieval and presentation

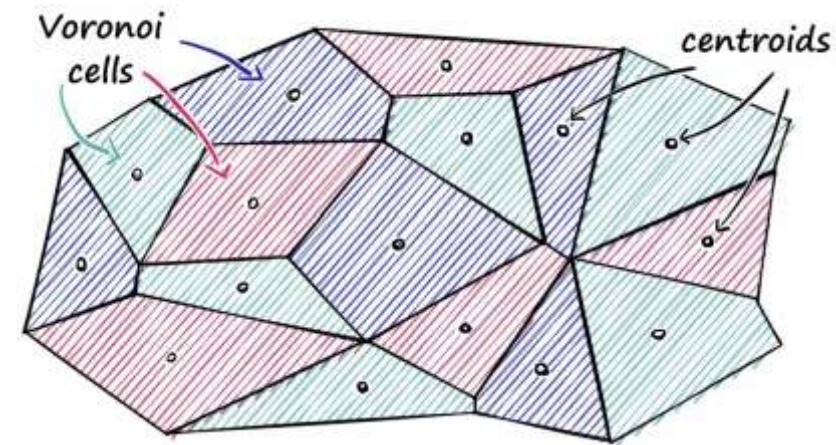


Index(-ing)

- The index aims to organize searchable items, such as documents and passages, to make them easier to locate
- Each IR method employs an index constructed during the indexing process to identify relevant information in response to a query

1	Term	Doc_1	Doc_2
2			
3	Quick		X
4	The	X	
5	brown	X	X
6	dog	X	
7	dogs		X
8	fox	X	
9	foxes		X
10	in		X
11	jumped	X	
12	lazy	X	X
13	leap		X
14	over	X	X
15	quick	X	
16	summer		X
17	the	X	
18			

Inverted index for sparse retrieval

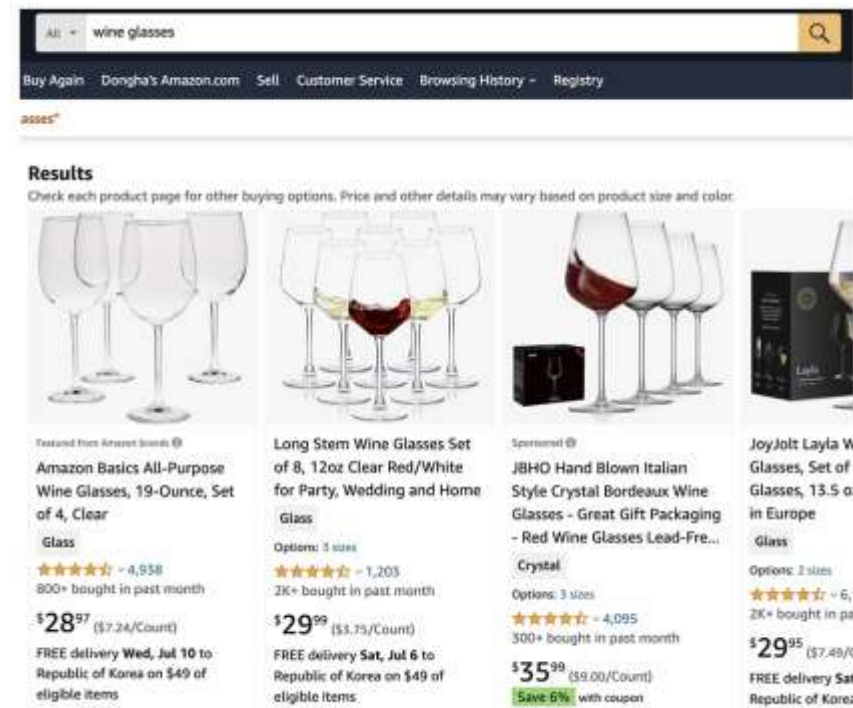
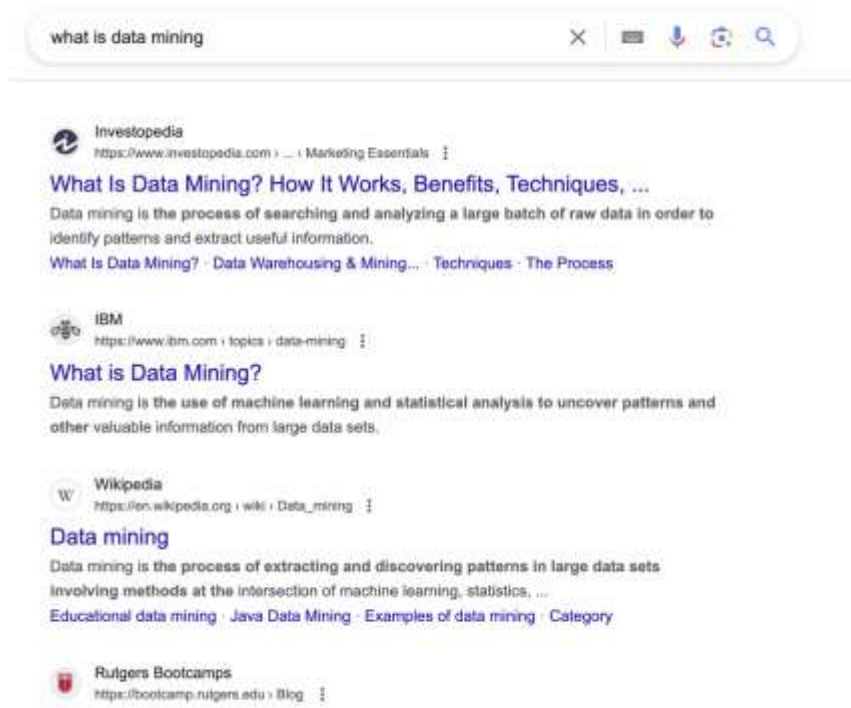


FAISS index for dense retrieval

[source] Elasticsearch official Documentation

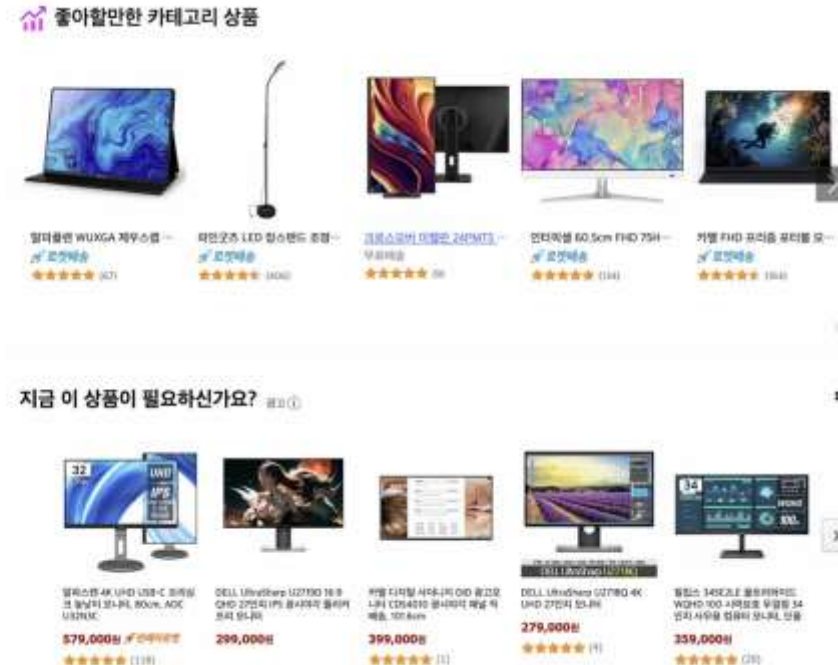
Application of Information Retrieval

- Web Search & Item Search
 - Search engines (e.g., Google, Naver, ...)
 - E-commerce (e.g., Amazon, Coupang, ...)



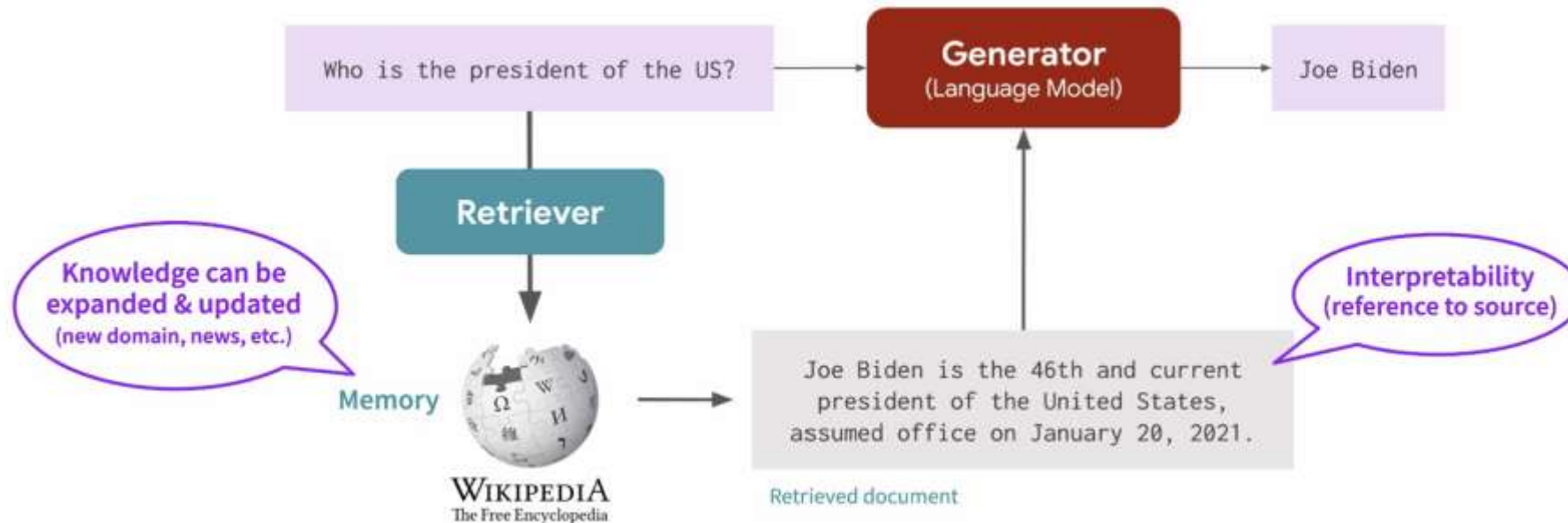
Application of Information Retrieval

- Recommender System
 - OTT Service: A user has watched preferred movies or TV series
 - E-commerce: A user has explored items that he/she wants



Application of Information Retrieval

- Retrieval-Augmented Generation (RAG)
 - Making large language models (LLMs) to leverage retrieved information (or knowledge) can enhance the reliability and reduce hallucination



Application of Information Retrieval

- Retrieval-Augmented Generation (RAG)
 - Given a query, more flexible and accurate results (i.e., direct answers with citations) can be provided by combining LLMs with web search
 - Perplexity serves “answer engine”, rather than “search engine”

최근 5년간 한국 AI R&D 정책의 방향성은 어떻게 바뀌었어

Perplexity 출처 · 8 단계

미래를 보는 창 - 전자신문
[ET시론]AI와 함께하는 산업기술 R&D 현황과 전망 - 전자신문

goover.ai
2025 한국 국방 AI 현황과 전략 리뷰: 정책·기술·산업 관점 -

spri.kr
AI Index 2025 주요내용 및 시사점 - SPRi - 소프트웨어정책연구소

www.elec.kdli.re
신도형 R&D와 AI 융합화로 미래 도약 준비 | KDI 경제교육·정보센터

최근 5년간 한국의 AI R&D 정책 방향성은 다음과 같이 변화해왔습니다.

주요 변화 흐름

- 산업 현장 중심 실용 AI로 전환
 - 과거에는 데이터 기반 의사결정 시스템, 스마트공장 제어, 수술용 증강현실(AR), 금융 의사결정 지원 등 다양한 산업 분야에 AI 연관 R&D를 집중 지원했습니다.
 - 최근에는 산업 현장에 바로 적용 가능한 실용 AI 기술, 즉 제품과 공정 특성에 최적화된 맞춤형 AI, 디지털 트윈, 소형화된 모델, 엣지(Edge) 기반 AI 처리 등으로 방향성이 옮겨가고 있습니다.
 - 특히 2024년부터는 생성형 AI가 산업과 일상 속으로 빠르게 확산되며, 신규 과제가 전년 대비 3배 증가하는 등 실증 기반, 경량화, 온디바이스(On-device) AI 시스템에 대한 수요와 지원이 크게 강화되고 있습니다.
- 정부 R&D 투자 및 예산 확대
 - 정부는 2020년부터 2030년까지 총 1조 원 규모의 AI 투자 계획을 발표했으며, 2025년도 정부 R&D 예산은 전년 대비 11.8% 증가한 29조 7천억 원으로 편성되었습니다.
 - AI 관련 신규 과제와 출연금도 꾸준히 증가하고 있습니다. 예를 들어, 2024년 신규 과제는 212건, 출연금은

oo.ai

최근 5년간 한국 AI R&D 정책의 방향성은 어떻게 바뀌었어?

| AI R&D 투자 확대 및 전략 기술 집중

AI 활용 연구 개발(R&D) 촉진
AI 인재 양성 및 생태계 조성
AI 규제 및 거버넌스 정비
데이터 활용 및 인프라 강화

AI R&D 투자 확대 및 전략 기술 집중

- 정부는 AI를 '게임 체인저'로 규정하고 AI 3대 강국 도약을 위해 향후 5년간 16조 원 이상을 투자할 계획이며, 민간 투자 100조 원 시대를 열겠다는 목표를 세웠습니다.
- AI를 포함한 12대 국가전략기술 분야에 대한 투자를 확대하고 있으며, 특히 AI, 바이오, 양자 기술을 "3대 게임체인저" 기술로 지정하여 집중적으로 투자하고 있습니다.
- AI 외에 차세대 통신, 양자 기술, 바이오 기술 등 미래 기술 경쟁력 확보를 위한 R&D 투자를 강화하고 있습니다.
- AI를 조세특례제한법상 국가전략기술로 지정하여 세액공제율을 높이는 방안을 추진하고 있습니다. 중소기업의 경우 R&D에 대한 세액공제율이 기존 30~40%에서 최대 40~50%까지 확대될 수 있습니다.
- 혁신도전형 R&D 투자 확대를 통해 실패 가능성이 높지만 성공 시 부가가치가 큰 분야에 대한 투자를 확대하고 있습니다.
- 2030년까지 최신 AI 반도체 기술 경쟁력 확보를 위해 민·관 합작으로 '국가 AI컴퓨팅센터' 구축을 추진하고 있습니다.

과학기술정보통신부

혁신도전형 R&D 성공·실패 평가 폐지...정부 "AI 등 R&D 집중 투자"

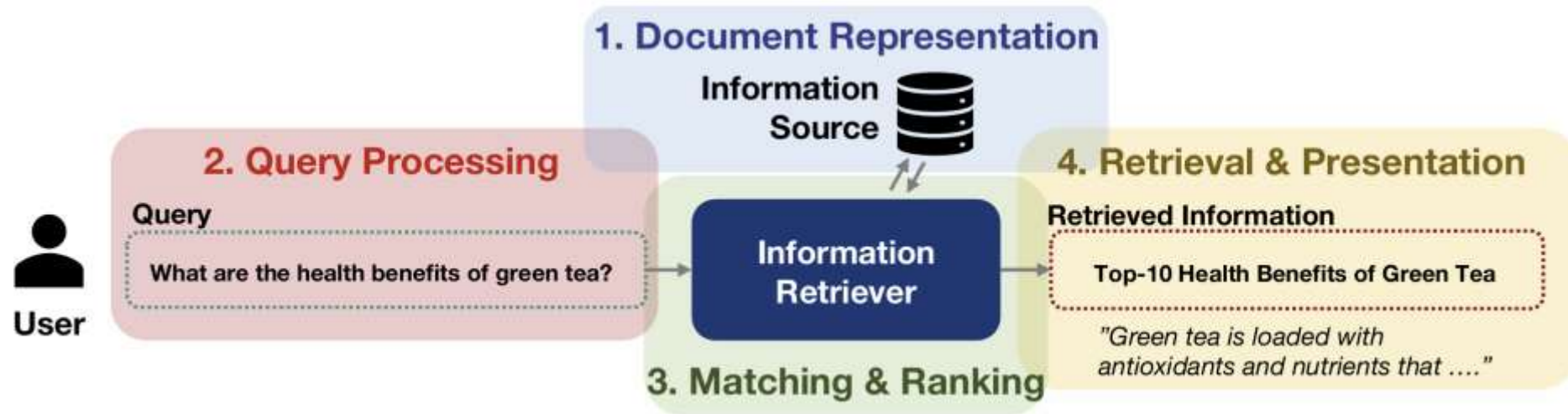
m.dongascience.com 2025-03-13

[현장] 류광준 과기부 혁신본부장 "혁신도전형 R&D 1조원 투자...AI 특사업 추진" - 이비엔(EBN)뉴스센터

이비엔(EBN)뉴스센터 2025-02-13

Information Retrieval

- Key Components of Information Retrieval
 1. Document representation
 2. Query processing
 3. Matching and ranking
 4. Retrieval and presentation

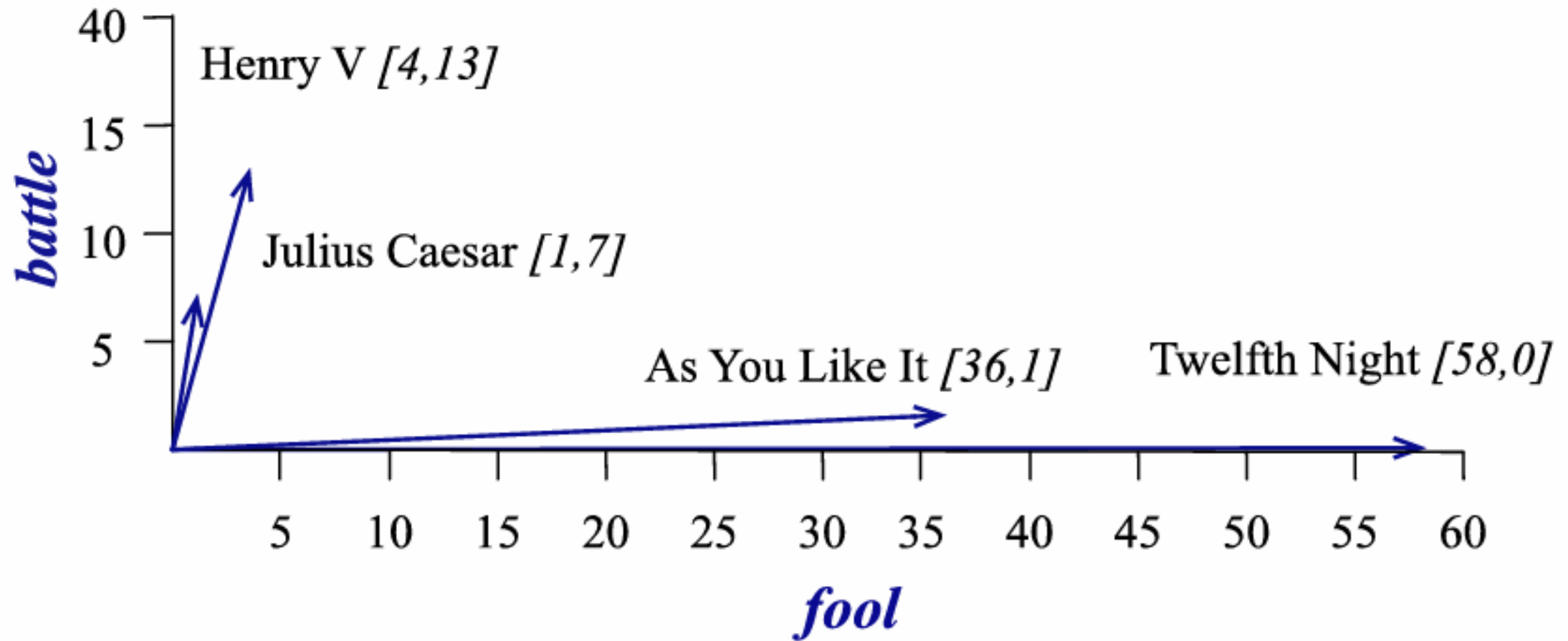


Term-Document Matrix

- Each document is represented by a vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizing Document Vectors



Web Search: Boolean → Ranked retrieval

- Boolean Retrieval (Old System)
 - Query–document match is **Boolean**:
→ Document either matches or does not.
- Strengths
 - Works for **expert users** (precise needs, domain knowledge).
 - Fine for applications (can handle 1000s of results).
- Weaknesses
 - Most users **cannot or will not** write Boolean queries.
 - Most users don't want to sift through thousands of results.
 - Especially problematic for web search.

Problem with Boolean Search: feast or famine

- Exact Match Queries
 - Often return results that are **too few (0)** or **too many (100k+)**
 - Example:
 - Query 1: "*COSE475 lecture notes*" → ~200,000 hits
 - Query 2: "*COSE475 lecture notes quiz2 solutions*" → 0 hits
- Challenge:
 - Designing queries that yield a **manageable number of results** requires skill.
- Boolean Pitfall
 - AND → overly restrictive (too few)
 - OR → overly broad (too many)

Ranked Retrieval Models

- Ranked retrieval
 - Boolean retrieval: returns a **set** of matching documents.
 - Ranked retrieval: returns an **ordered list** of documents by relevance.
 - Focus is not only whether a document matches, but how well it matches.
- Free-Text Queries
 - Classical IR: requires a formal query language (operators: AND, OR, NOT).
 - Free-text queries: user inputs natural language words/phrases.
 - Example:
 - Formal query → "information" AND "retrieval"
 - Free-text query → information retrieval

Scoring as the Basis of Ranked Retrieval

- **Goal:** return documents most useful to the searcher.
- **How?** → Rank documents with respect to the query.
- Assign a relevance score (e.g., $\in [0, 1]$) to each document.
- Score reflects degree of match between query and document.

Query-Document Matching Scores

- Scoring Query–Document Pairs
 - Need a function: **score(query, document)**
 - Start simple: one-term query
- Intuition
 - If query term **absent** $\rightarrow$ score = 0
 - If query term **present** $\rightarrow$ score $\uparrow$ with frequency

Documents as Vectors

- Vector Space Model
 - Represent collection in a $|V|$ -**dimensional space**
 - $|V|$ = vocabulary size
 - **Axes = terms**
 - **Documents = vectors (or points)**
- Properties
 - Very high-dimensional (web search → tens of millions).
 - Sparse vectors: most entries = 0.

Queries as Vectors

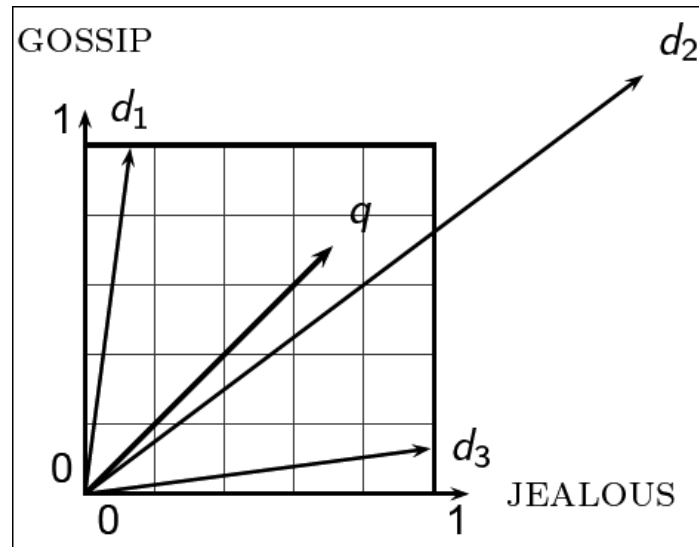
- Key Ideas
 - Queries as vectors in the same space as documents.
 - Rank documents by their **proximity** to the query vector.
- Proximity
 - Defined as **vector similarity**.
 - Intuition: proximity $\approx$ inverse of distance.

Formalizing Vector Space Proximity

- First Attempt
 - Define proximity as distance between two vectors
 - Natural choice: **Euclidean distance**
- Problem with Euclidean Distance
 - Sensitive to **vector length**
 - Two documents with same content but different length
 - large distance
 - Not a good measure of similarity

Why Distance is a Bad Idea

- Problem
 - **Euclidean distance** between query $\vec{q}$ and document $\vec{d}_2$ is **large**
 - Even though their term distributions are very similar
 - Long documents $\rightarrow$ longer vectors
 - Distance grows with **vector length**, not just with **content similarity**

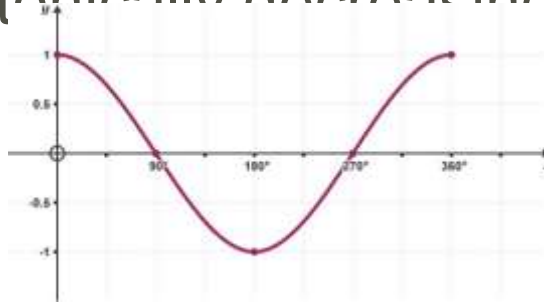


Use Angle instead of Distance

- Thought Experiment
 - Take a document d and append it to itself $\rightarrow$ new longer doc d'
 - Semantically: d and d' are the same
 - Problem: Euclidean distance(d, d') = large
 - Solution: Angle(d, d') = 0 $\rightarrow$ **maximal similarity**
- Key Idea
 - Rank documents by their **angle** with the query vector
 $\rightarrow$ Equivalent to **cosine similarity**

From Angles to Cosines

- Equivalence
 - Rank docs by smaller angle with query $\Leftrightarrow$ Rank docs by larger cosine with query
 - Because $\cos(\theta)$ is monotonically decreasing on $[0^\circ, 180^\circ]$



- Why Cosines?
 - Angles are intuitive, but **hard to compute directly** in high dimensions
 - Cosine is mathematically simple:

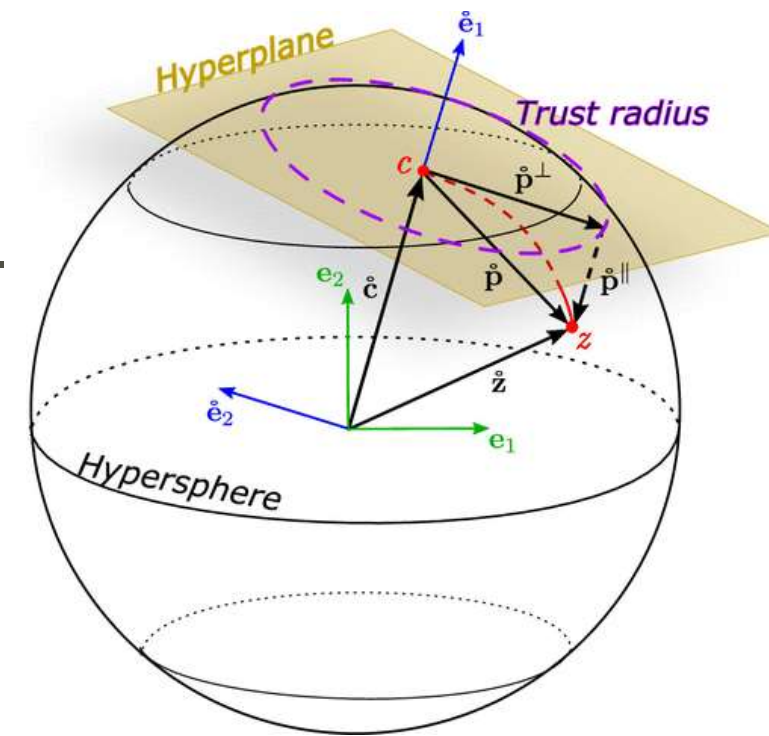
$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|}$$

Length Normalization

- A vector can be (length-) normalized by dividing each of its components by its length → for this we use the **L2 norm**:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Resulting vector = **unit length** (lies on unit hypersphere).
- Documents d and d' (d appended to itself)
 - Different raw lengths
 - Identical vectors after normalization.



Cosine(Query, Document)

- **Formula**

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\boxed{\vec{q}} \cdot \boxed{\vec{d}}}{\boxed{|\vec{q}|} \cdot \boxed{|\vec{d}|}} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

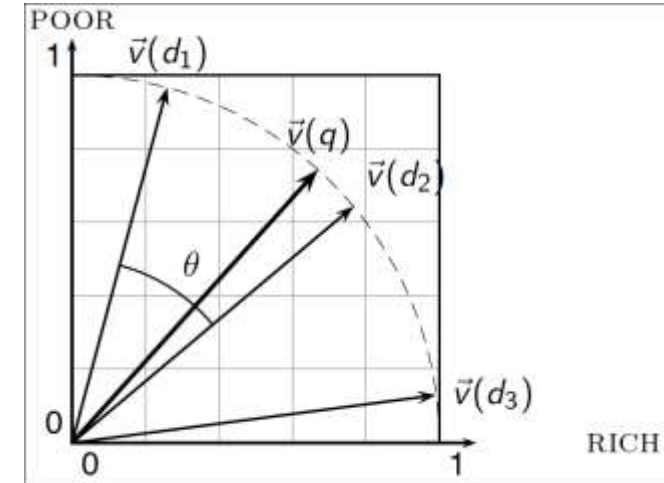
Diagram annotations: A red arrow labeled "Dot Product" points to the dot operator in the first fraction. Two red arrows labeled "Unit Vectors" point to the boxed vectors $\vec{q}$ and $\vec{d}$ in the second fraction.

- q_i is the tf-idf weight of term i in the query
 - d_i is the tf-idf weight of term i in the document
 - **Dot product** in numerator → captures overlap between query & document.
 - Normalization(**Unit Vectors**) in denominator → removes sensitivity to length.
- $\cos(\vec{q}, \vec{d})$ = cosine similarity of $\vec{q}$ and $\vec{d}$.
 - Equivalently, the cosine of the angle of $\vec{q}$ and $\vec{d}$.

Cosine for Length-Normalized Vectors

- After **L2 normalization**, each vector has **unit length**.
- Cosine similarity reduces to the **dot product**:

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$



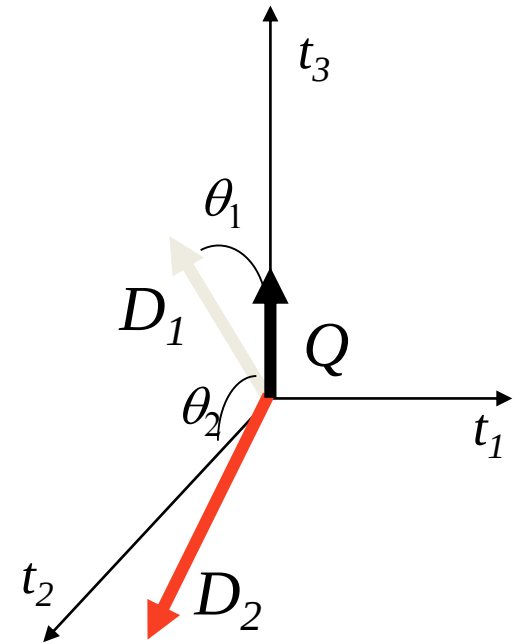
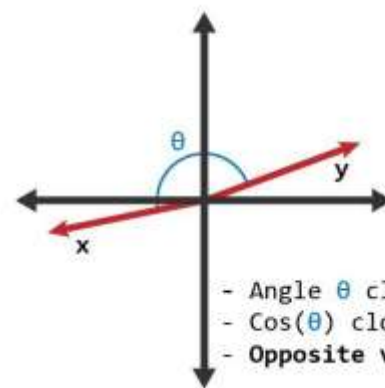
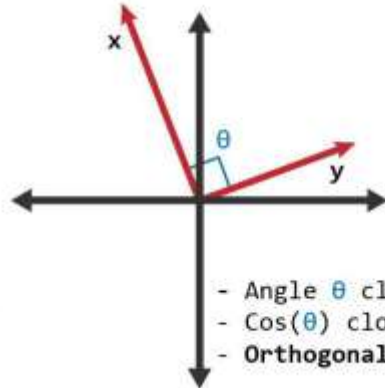
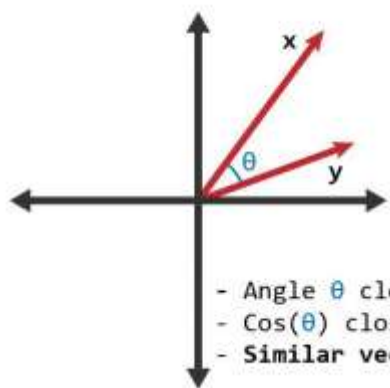
- No denominator needed ($\|q\| = \|d\| = 1$).
- Cosine similarity = **weighted overlap** of query & document terms.
- Efficient to compute in high-dimensional, sparse spaces.

In Summary: Cosine Similarity

- **Cosine similarity** measures how similar two vectors are by computing the cosine of the angle between them.

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}}$$

- $\mathbf{q}$: query vector
- $\mathbf{d}$: document vector
- q_i, d_i : the i -th term in each query and document vector



Example: Cosine Similarity Among 3 Documents

- Measure **similarity** between three novels:
 - **SaS**: Sense and Sensibility
 - **PaP**: Pride and Prejudice
 - **WH**: Wuthering Heights

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term Frequencies (counts)

- Note: To simplify this example, we don't do idf weighting.

3 Documents Example (continued)

- Step 1: Log Frequency Weighting
 - Raw term counts are compressed using **log transform**.
- Step 2: Length Normalization
 - **Normalize vectors** so that document length doesn't dominate.

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

3 Documents Example (continued)

Step 1. Log Frequency Weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

Step 2. Length Normalization

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

- **Step 3: Cosine Similarity**

- $\cos(\text{SaS}, \text{PaP})$
 $\approx 0.789 \times 0.832 + 0.515 \times 0.555 + 0.335 \times 0.0 + 0.0 \times 0.0 \approx 0.94$
- $\cos(\text{SaS}, \text{WH}) \approx 0.79$
- $\cos(\text{PaP}, \text{WH}) \approx 0.69$

Computing Cosine Scores

COSINESCORE(q)

```
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[ $d$ ] + =  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[ $d$ ] = Scores[ $d$ ] / Length[ $d$ ]
10 return Top  $K$  components of Scores[]
```


Sparse Retrieval

- Sparse Retrieval is a traditional IR approach that relies on the **exact matching of terms** (i.e., **lexical similarity**) between queries and documents.
- It typically involves representing documents and queries as **sparse vectors**, where each dimension corresponds to a unique term from the vocabulary.
 - TF-IDF and BM25 are the two commonly-used techniques.
- Relevance is computed by the **cosine similarity** between the query vector and document vectors”

Bag of Words Model

- Vector representation that does not consider the ordering of words in a document.
 - *John is quicker than Mary* and *Mary is quicker than John* have the same vectors
- This is called the **bag of words** model.
 - The term-document matrix.



[slide courtesy: Stanford, CS276]

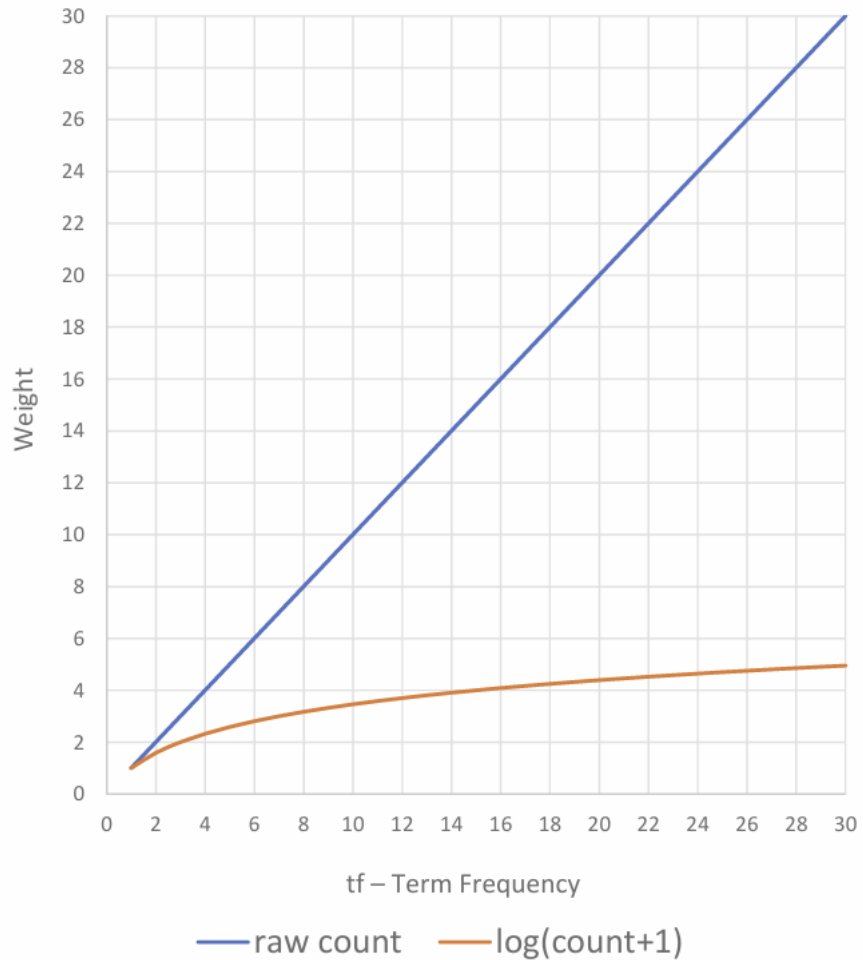
[source: <https://github.com/sebastian-hofstaetter/teaching>]

Term Frequency (TF)

- The term frequency $tf_{t,d}$ of term t in document d is defined as the number of times that t occurs in d .
 - Note: Frequency means count in IR
- We want to use TF when computing query-document match scores. But how?
- Raw term frequency is not what we want:
 - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
 - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

[slide courtesy: Stanford, CS276]

Term Frequency & Logarithm



- In long documents, a term may appear hundred of times.
- Retrieval experiments show that using the logarithm of the number of term occurrences is more effective than raw counts.
- Commonly used approach:
apply logarithm

$$\log(1 + tf_{t,d})$$

[source: <https://github.com/sebastian-hofstaetter/teaching>]

Document Frequency (DF)

- df_t = how many documents does a term t appear in.
- Rare terms are more informative than frequent terms.
 - Recall stop words (“*the*”, “*a*”, “*to*”, “*of*”, ..)
- Consider a term in the query that is rare in the collection.
 - e.g., TUWIEN in a news corpora
- A document containing this term is very likely to be relevant to the query TUWIEN.
→ We want a high weight for rare terms like TUWIEN..

[source: <https://github.com/sebastian-hofstaetter/teaching>]

Inverse Document Frequency (IDF)

- A common way of defining the inverse document frequency of a term is as follow
- $idf(t) = \log \frac{|D|}{df_t}$
 - $|D|$: Total # of documents
 - df_t : # of Documentes with > 0
- df_t is an inverse measure of the “informativeness” of the term.
- $df_t \leq |D|$
- Logarithm is used also for idf to "dampen" its effect.

[source: <https://github.com/sebastian-hofstaetter/teaching>]

TF-IDF

- $\text{TF-IDF}(q, d) = \sum_{t \in T_d \cap T_q} \log(1 + tf_{t,d}) * \log\left(\frac{|D|}{df_t}\right)$
 - $\sum_{t \in T_d \cap T_q}$: Sum over all query terms, that are in the index
 - $tf_{t,d}$: Term frequency
 - $|D|$: Total # of documents
 - df_t : # of documents with > 0
- TF $\rightarrow$ increases with the number of occurrences within a document
- IDF $\rightarrow$ increases with the rarity of the term in the collection
- A rare word (in the collection) appearing a lot in one document creates a high score
- Common words are downgraded

[source: <https://github.com/sebastian-hofstaetter/teaching>]

BM25

- BM25 (BestMatch25) scoring function combines TF and IDF components to compute the relevance score of a document d for a given query q
- BM25 enhances the traditional TF-IDF approach by incorporating (1) **term frequency saturation** and (2) **document length normalization**

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{TF}(t, d) \cdot (k_1 + 1)}{\text{TF}(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}$$

- k_1 controls term frequency saturation
- b controls document length normalization.

BM25

- BM25 (BestMatch25) scoring function combines TF and IDF components to compute the relevance score of a document d for a given query q
- BM25 enhances the traditional TF-IDF approach by incorporating (1) **term frequency saturation** and (2) **document length normalization**

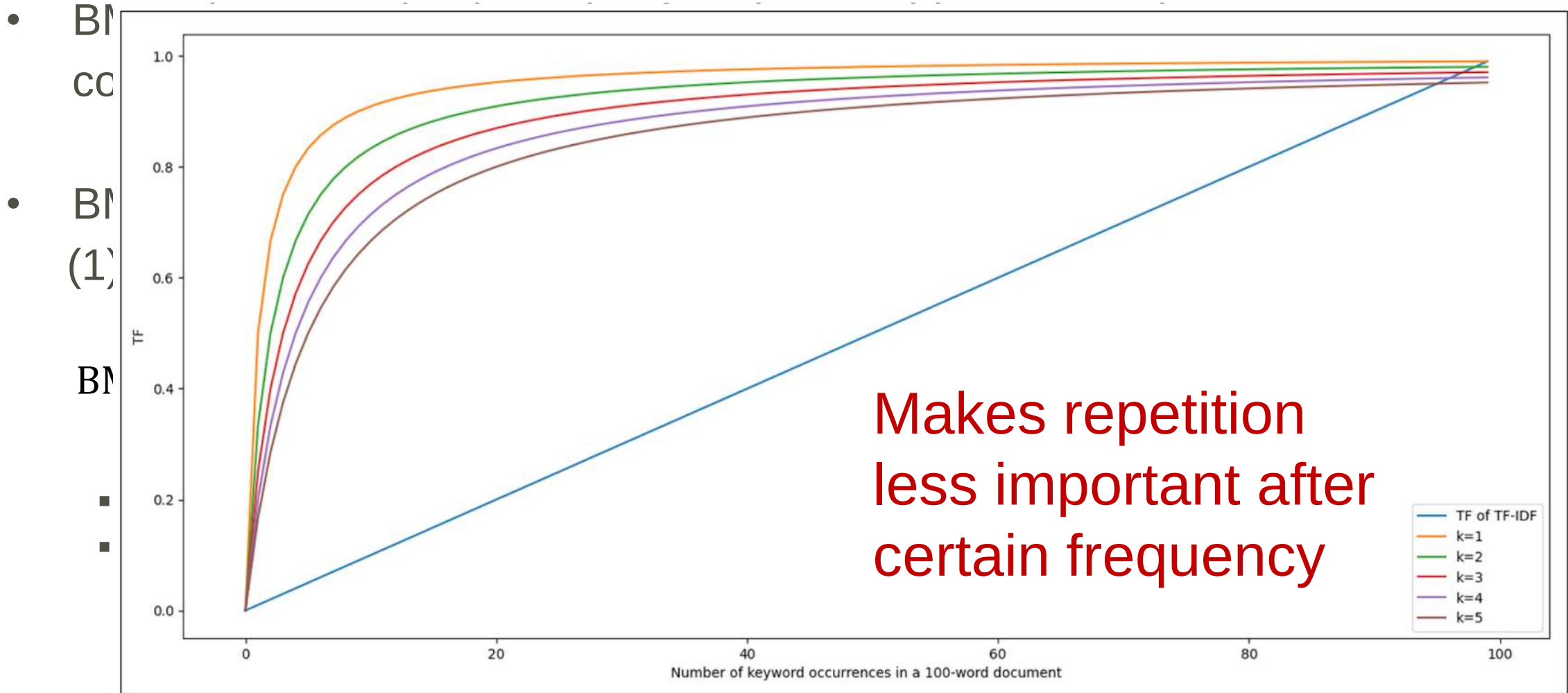
$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{TF}(t, d) \cdot (k_1 + 1)}{\text{TF}(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}$$

- k_1 controls term frequency saturation
- b controls document length normalization.

TF saturation



BM25



<https://zilliz.com/learn/mastering-bm25-a-deep-dive-into-the-algorithm-and-application-in-milvus>

BM25

- BM25 (BestMatch25) scoring function combines TF and IDF components to compute the relevance score of a document d for a given query q
- BM25 enhances the traditional TF-IDF approach by incorporating (1) **term frequency saturation** and (2) **document length normalization**

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{TF}(t, d) \cdot (k_1 + 1)}{\text{TF}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

- k_1 controls term frequency saturation
- b controls document length normalization.

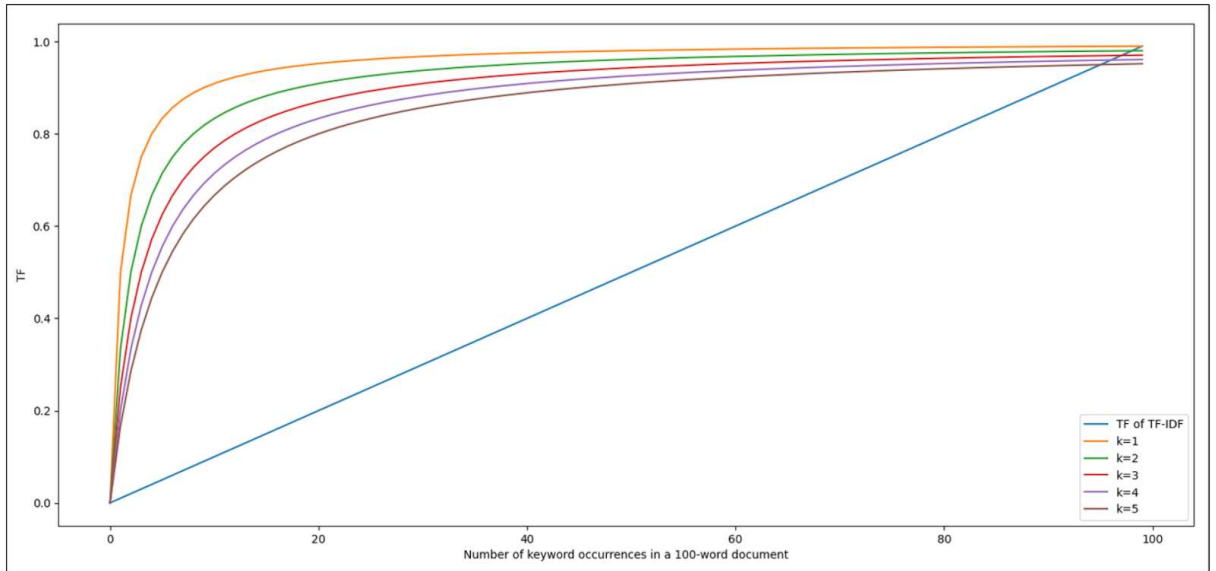
Document length normalization

BM25

- BM25 (BestMatch25) scoring function computes the relevance score of a document
- BM25 enhances the traditional TF-IDF with (1) **term frequency saturation** and (2)

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{TF}(t, d) \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}{\text{TF}(t, d) + k_1}$$

- k_1 controls term frequency saturation
- b controls document length normalization.



Document length normalization:
Makes saturation factor vary
depending on the document length

Robertson et al., Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval, SIGIR'94
<https://zilliz.com/learn/mastering-bm25-a-deep-dive-into-the-algorithm-and-application-in-milvus>

BM25 vs. TF-IDF

- Simple case of BM25 looks a lot like TF-IDF
- 1 main difference: BM25 $tf_{t,d}$ component contains saturation function
 - Therefore works better in practice
- BM25 variants can be adapted to:
 - Incorporate additional reference information
 - Long(er) queries
 - multiple field

[source: <https://github.com/sebastian-hofstaetter/teaching>]

Sparse Retrieval

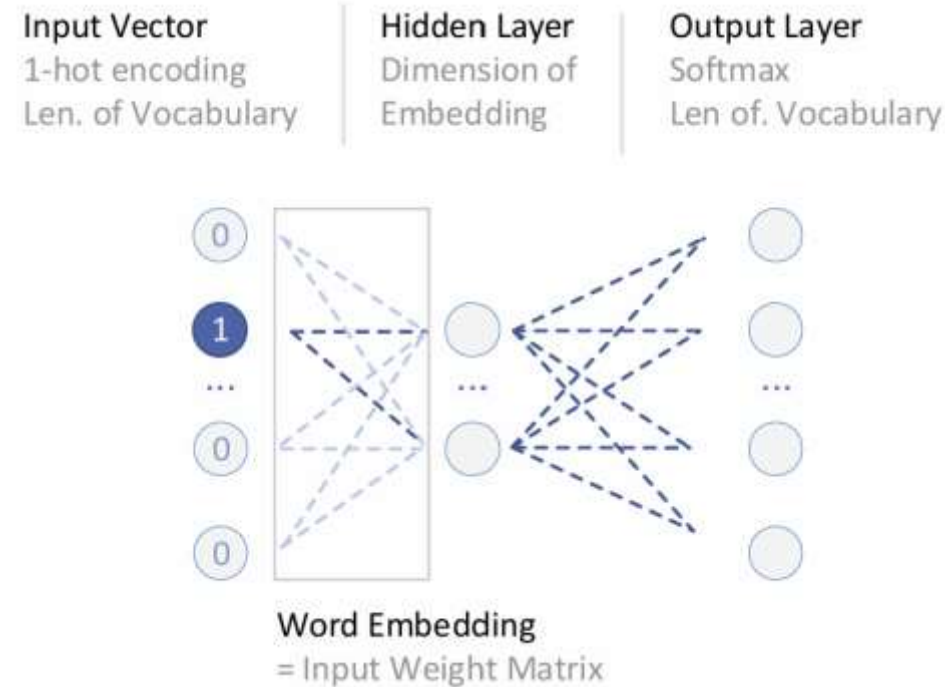
- Advantages of Sparse Retrieval
 - **Simplicity:** Relatively straightforward to implement and understand
 - **Efficiency:** Inverted index allows for fast lookups and efficient query processing
 - **Transparency:** The retrieval results are usually interpretable, based on term matching
- Disadvantages of Sparse Retrieval
 - **Limited semantic understanding:** In the example below, "bad guy" is used in the query, but "villain" is used in the information source (i.e., passage)
 - Query: "Who is the **bad guy** in lord of the rings?"
 - Relevant Information: "Sala Baker is best known for portraying **the villain** Sauron in the Lord of the Rings trilogy"
 - Lexical similarity-based methods, such as TF-IDF and BM25, cannot effectively capture the relevance because different terms can have the same meaning

Neural Information Retrieval

- Neural methods have significantly advanced IR by enabling semantic matching **beyond exact keyword matching**.
- In particular, **word embeddings** allow for comparing terms based on their semantic similarity rather than surface form.
- Beyond individual words, deep neural architectures such as CNNs, RNNs, and Transformers can be used to encode entire sentences or documents into **dense vector representations**.
- These embeddings capture contextual and semantic information, enabling dense retrieval in high-dimensional space.

Neural Word Embeddings - Word2vec

- Input and output words are represented as sparse “one hot” encodings.
- Hidden layer learns a dense encoding of the input word that allows it to predict its neighbors.
- Two algorithms
 1. **Skip-grams (SG)**
 - Predict context words given target (position independent)
 2. **Continuous Bag of Words (CBOW)**
 - Predict target word from bag-of-words context



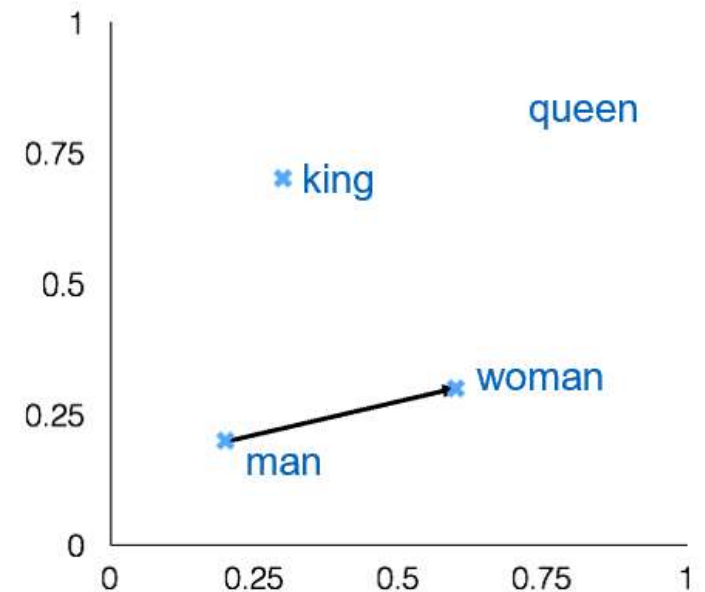
[slide courtesy: Texas at Austin, CS371R]

[source: <https://github.com/sebastian-hofstaetter/teaching>]

Mikolov al., Efficient Estimation of Word Representations in Vector Space, ICLR workshop 2013

Word2vec

- These representations are very good at encoding similarity and dimensions of similarity!
- Analogies testing dimensions of similarity can be solved quite well just by doing vector subtraction in the embedding space
 - Syntactically
 - $x_{apple} - x_{apples} \approx x_{car} - x_{cars} \approx x_{family} - x_{families}$
 - Similarly for verb and adjective morphological forms
 - Semantically
 - $x_{shirt} - x_{clothing} \approx x_{chair} - x_{furniture}$
 - $x_{king} - x_{man} \approx x_{queen} - x_{woman}$
- Limitation
 - Word ordering
 - Multiple senses per word

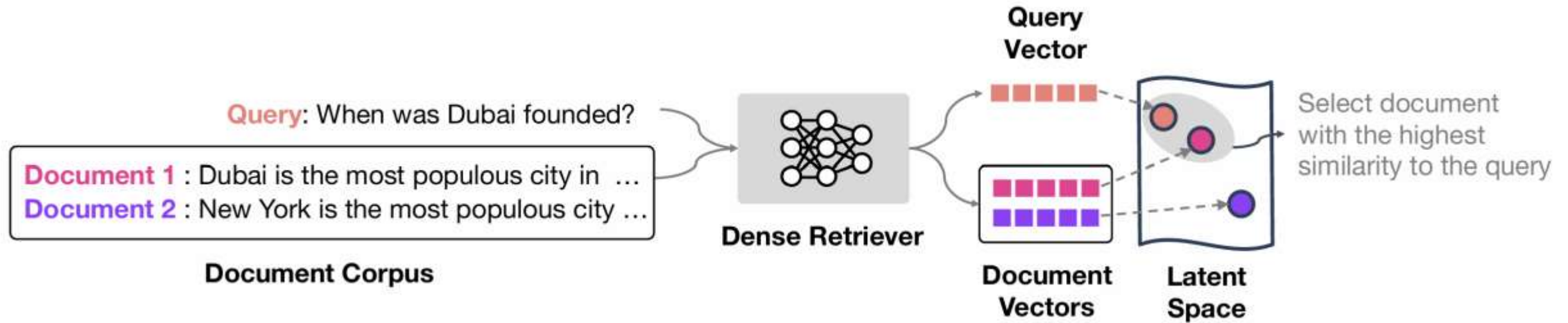


[slide courtesy: Texas at Austin, CS371R]

Mikolov et al., Efficient Estimation of Word Representations in Vector Space, ICLR workshop 2013

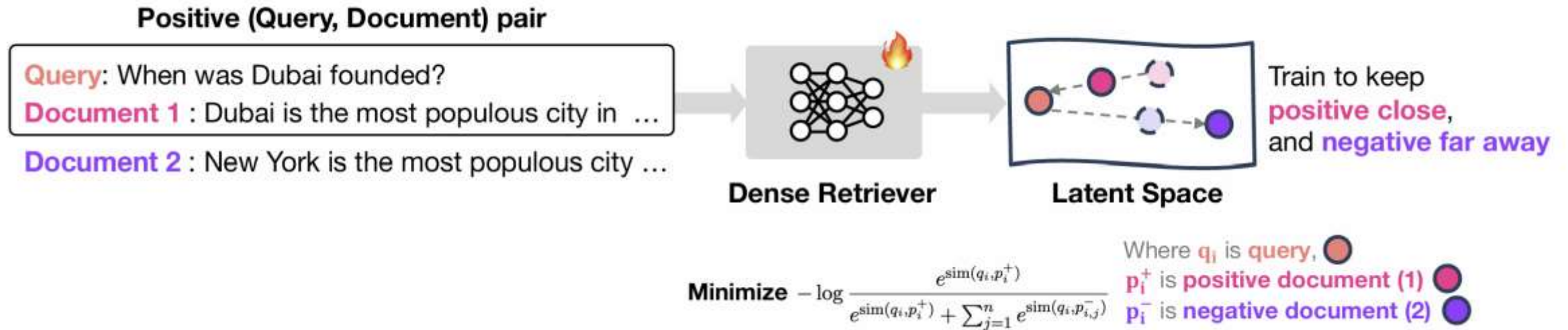
Dense Retrieval – Bi-encoder

- Dense retrieval utilizes **dense vectors** (also known as embeddings) to represent both queries and documents.
 - It focuses on capturing **semantic similarity** between queries and documents.
- Dense retrieval models are trained via **contrastive learning**, which encourages the query to be close to the positive document while pushing it away from negative documents.



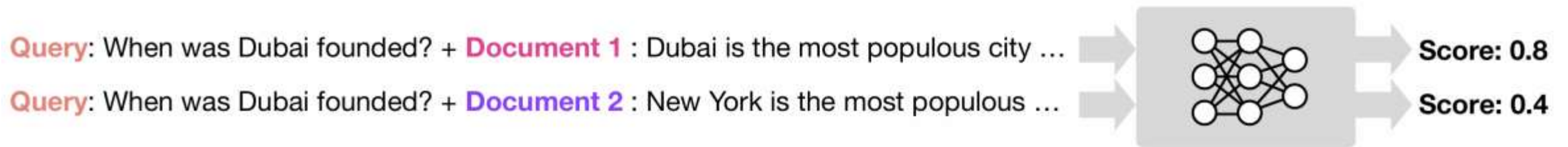
Dense Retrieval – Bi-encoder

- Dense retrieval utilizes **dense vectors** (also known as embeddings) to represent both queries and documents.
 - It focuses on capturing **semantic similarity** between queries and documents.
- Dense retrieval models are trained via **contrastive learning**, which encourages the query to be close to the positive document while pushing it away from negative documents.



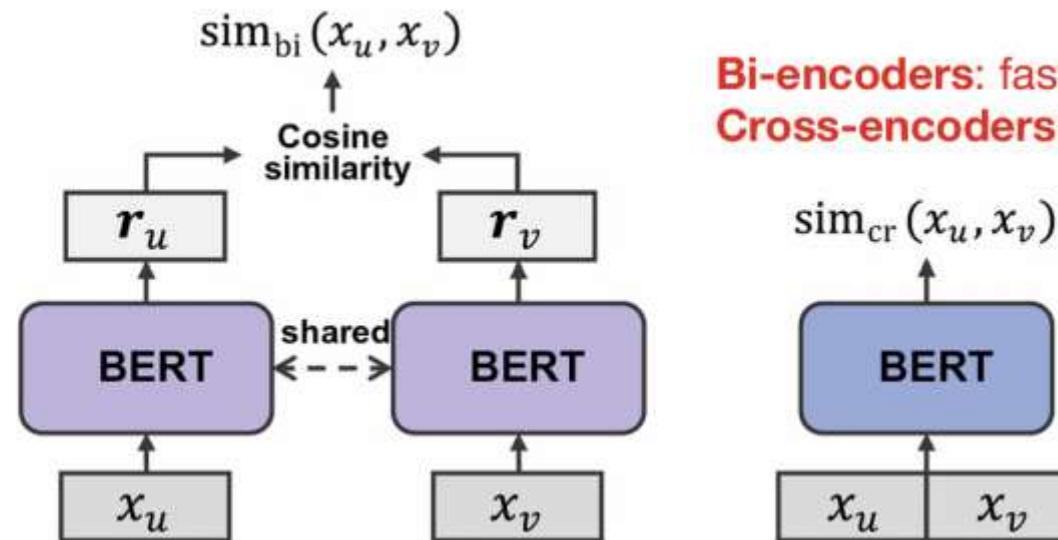
Dense Retrieval – Cross-encoder

- Cross-encoder **architecture combines two texts into a single sequence**, such as “query [SEP] document”, which is fed together into a single encoder.
- Self-attention lets **all query and document tokens interact fully**, enabling higher accuracy than bi-encoders.
- However, because every query-document pair must be input to the model individually, the computational cost is very high, and processing is slow.



Bi-encoder vs. Cross-encoder

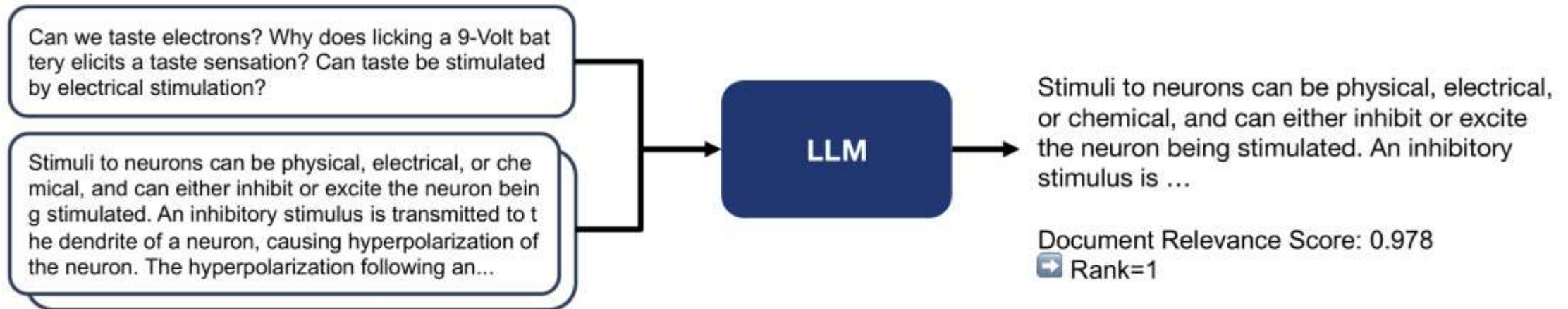
- Bi-encoder encodes two sentences separately
 - It's extremely fast and great for searching large databases but is less accurate
- Cross-encoder processes a pair of sentences together
 - It's very slow but highly accurate because it captures detailed interactions



Bi-encoders: fast initial search
Cross-encoders: re-rank the top results

Relevance Prediction (Ranking) with LLM

- LLMs can serve as powerful rankers; extension of the cross-encoder architecture.
- The **LLM analyzes the interaction via step-by-step reasoning** between a user query and a document by jointly processing them and **outputs a relevance score**.



Training Dense Retrievers

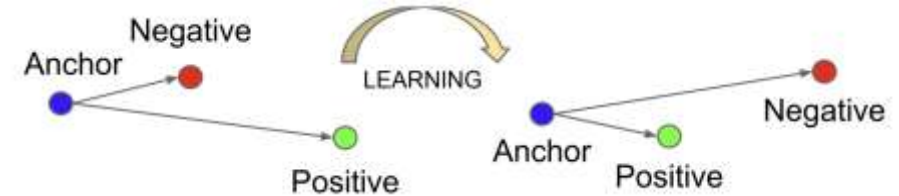
- Datasets
 - Query-document paired datasets are required for supervised training.
 - In a dataset with relevant document annotated for each query, the annotated documents are called positive documents and all the other documents the negatives.
- Loss function
 - The main idea is to put positive document embeddings close to the query embedding whereas pushing away negative document embeddings.
 - Triplet and contrastive losses are mainly used losses for training a retriever.

Triplet Loss

- Learn an embedding function $f(x)$ such that:
 - Same identity $\rightarrow$ close together
 - Different identities $\rightarrow$ far apart (by margin α)
- Formula

$$\mathcal{L}_{\text{triplet}} = \sum_i [\|f(x_a^i) - f(x_p^i)\|_2^2 - \|f(x_a^i) - f(x_n^i)\|_2^2 + \alpha]_+$$

- x_a : Anchor sample
 - x_p : Positive (same class)
 - x_n : Negative (different class)
 - $f(x)$: embedding function mapping input to feature space
 - α : margin hyperparameter
 - $[z]_+ = \max(0, z)$
- Pull anchor–positive closer
 - Push anchor–negative farther (by at least α)
 - Hard / semi-hard negative mining $\rightarrow$ crucial for effective training & faster convergence.



Schroff et al., FaceNet: A Unified Embedding for Face Recognition and Clustering, CVPR 2015

Contrastive Loss

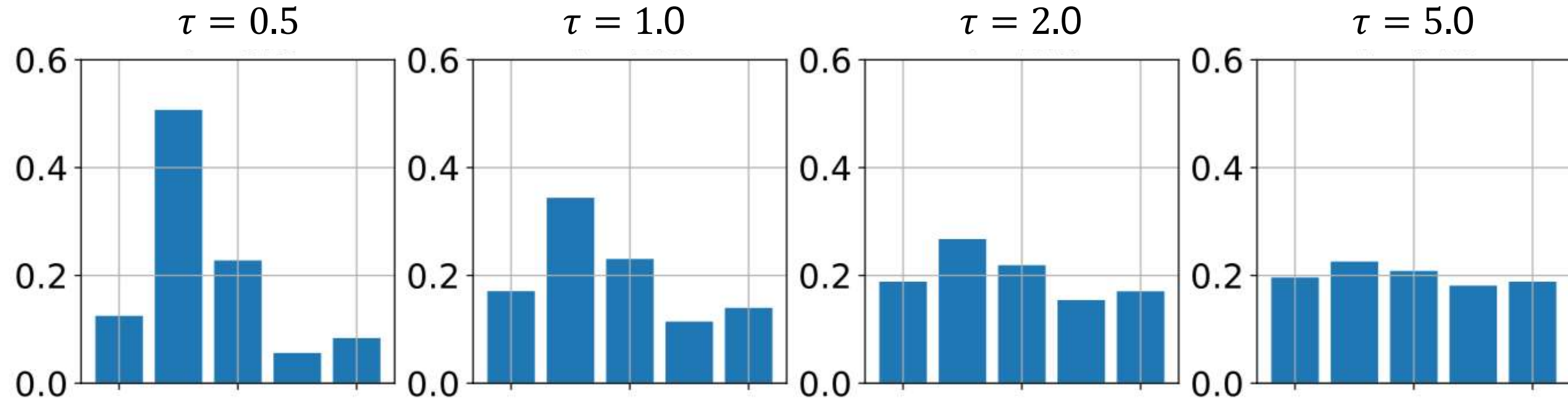
- Formula

$$\ell_i = -\log \frac{\exp(\text{sim}(q, z_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(q, z_j)/\tau)}$$

- q : projected embedding of the input query
 - z_i : projected embedding of sample i
 - z_j : embedding of any sample j in the batch
 - N : total number of samples in the batch
 - τ : temperature parameter (scales similarity sharpness)
 - $\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}$: cosine similarity
- Normalize similarities with softmax $\rightarrow$ probability.
 - Contrastive loss $\approx$ log-likelihood of picking the true positive among all candidates.
 - Encourages representations where semantically similar inputs cluster, while dissimilar inputs spread out.

Temperature Scaling in SoftMax

$$\ell_i = -\log \frac{\exp(x_i/\tau)}{\sum_{j=1}^N \exp(x_j/\tau)}$$



<https://www.themoonlight.io/ko/review/analytical-softmax-temperature-setting-from-feature-dimensions-for-model-and-domain-robust-classification>



Dense Retrieval

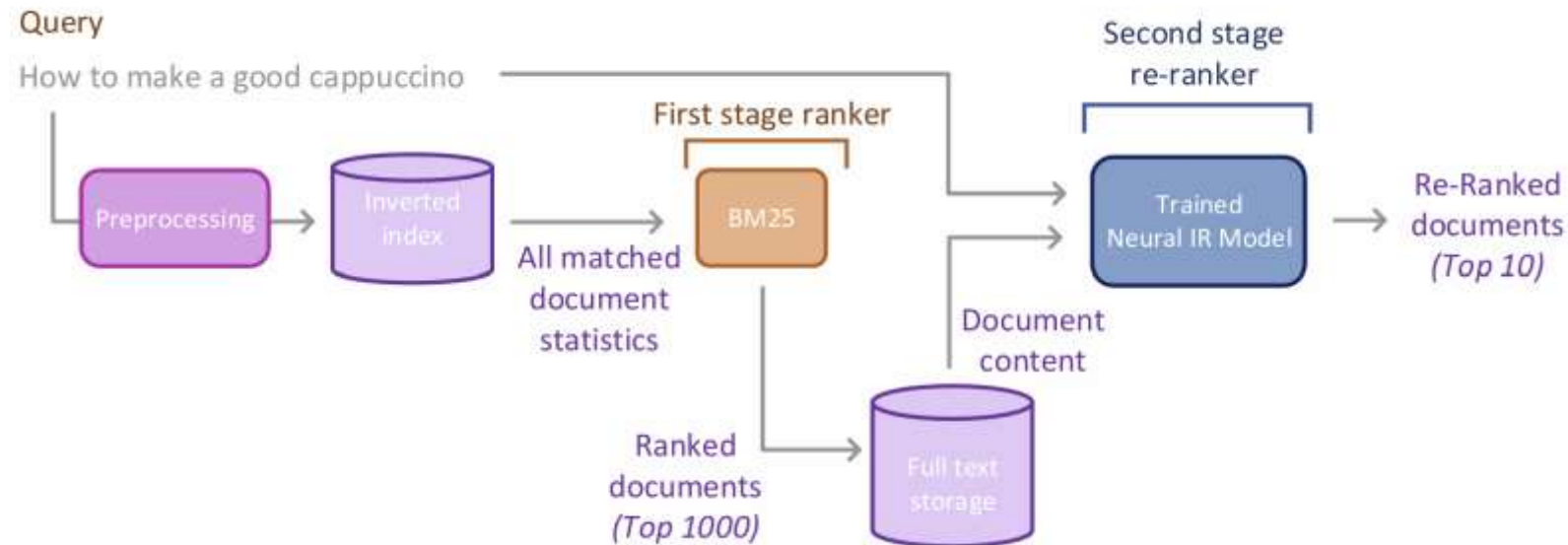
- Advantages of Dense Retrieval
 - **Semantic understanding:** Dense retrieval can handle synonyms and different phrasings more effectively; i.e., it captures the semantic meaning of terms.
 - **Rich query representation:** Dense representations can capture complex queries and long tail search queries better.
- Disadvantages of Dense Retrieval
 - **High computational cost:** Training models requires substantial computational resources and time, and inference can also be resource-intensive.
 - **Limited transparency:** Dense retrieval models can act as "black boxes," making it challenging to interpret why certain documents are retrieved.
 - **Dependency on data and models:** The effectiveness of dense retrieval is heavily affected by the quality of training data or changes in the model.

Contriever: Unsupervised Dense Retriever

- Contriever is a fully **unsupervised** dense retriever trained using **contrastive learning** on large-scale raw text corpora.
 - Unlike supervised methods like DPR, it requires **no manually labeled query-document pairs** during training.
- **Unsupervised Learning:**
 - Trained without any annotated datasets. Contriever generates positive pairs automatically based on semantic similarity.
- **Contrastive Learning Objective:**
 - Encourages semantically similar pairs to be close in embedding space, while dissimilar ones are pushed apart.
- **Positive Pair Generation Techniques:**
 - Inverse Cloze Task (ICT): Mask a sentence and use the masked + original as a training pair
 - Independent Cropping: Extract two different sentences from the same passage as a positive pair

Multi-Stage Ranking Pipeline

- The multi-stage ranking pipeline **narrows down the candidate pool** and increases **ranking accuracy** across multiple stages
 - In the first stage (a.k.a. **candidate generation**), a fast, simpler model (e.g., BM25, bi-encoder) is used to broadly identify potentially relevant candidate documents
 - In the second stage (a.k.a. **re-ranking**), more computationally expensive but accurate models are applied (e.g., cross encoder, LLM) only to a smaller number of candidates

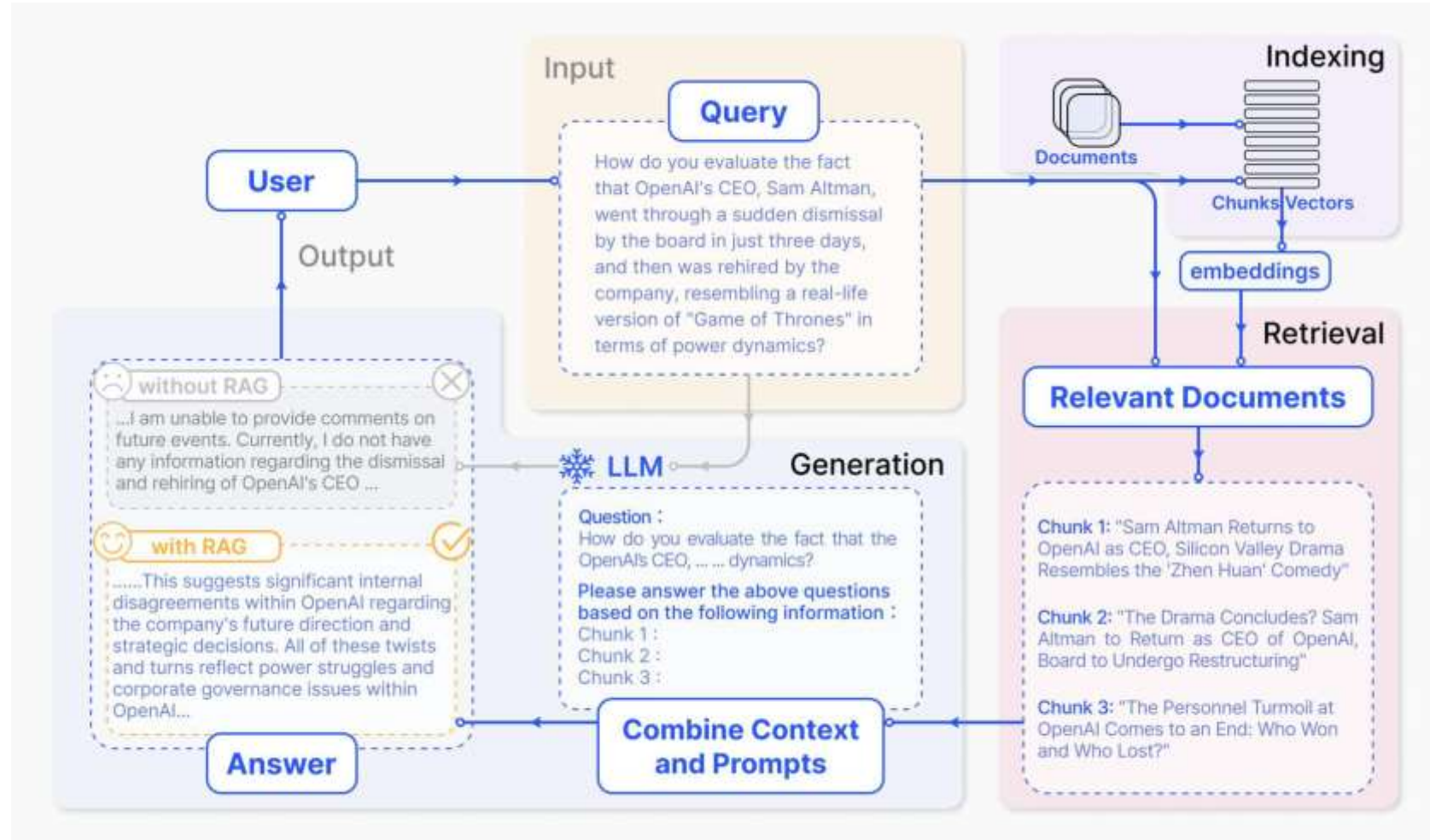


[source: <https://github.com/sebastian-hofstaetter/teaching>]

Retrieval Augmented Generation

Retrieval Augmentation

- W/o RAG
 - Generate texts directly from the query.
 - *LLMs should contain everything.*
- W/ RAG
 - Retrieve relevant documents first.
 - Generate an output text given the retrieved documents.
 - *No need to retain entire knowledge within the model.*



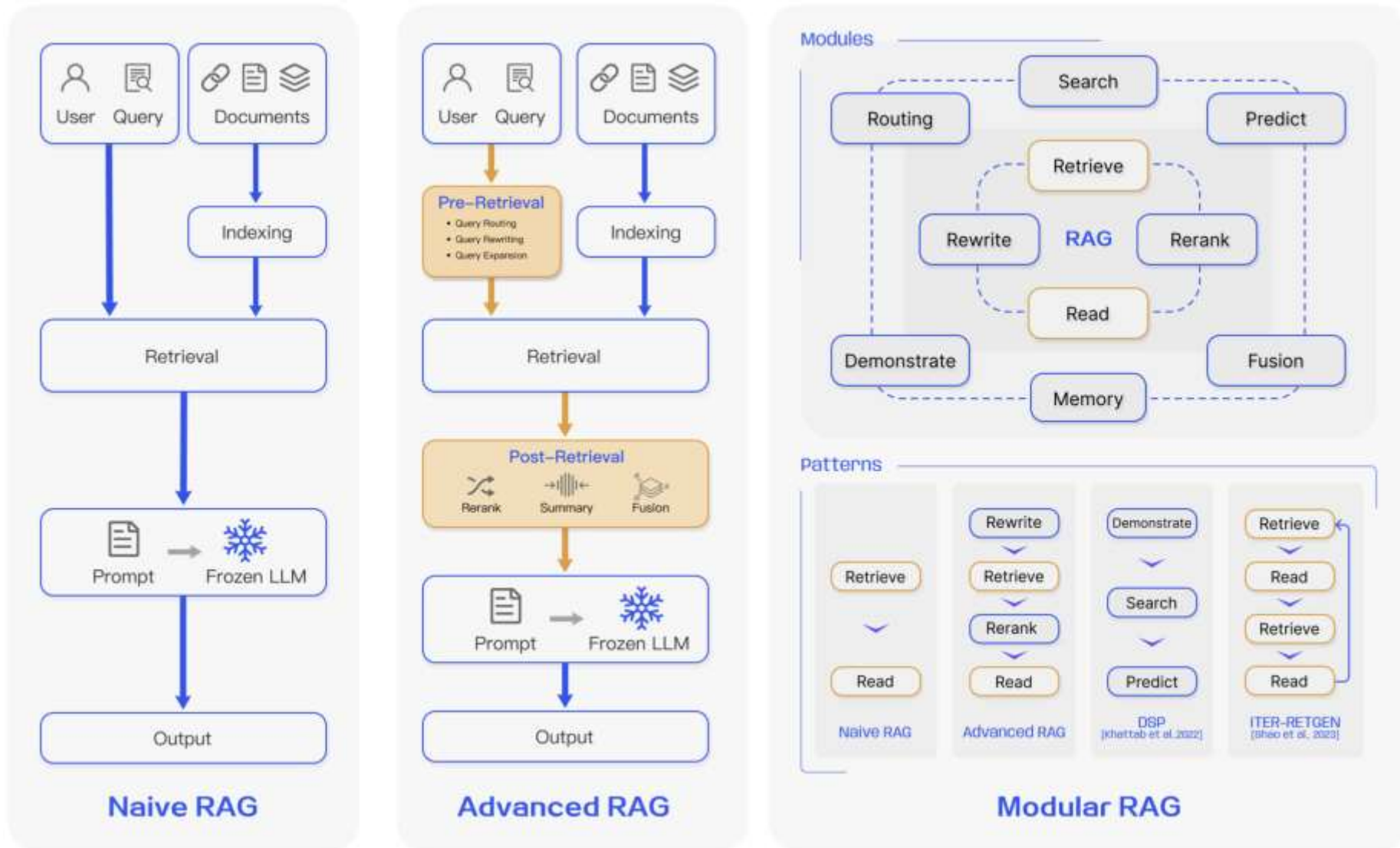
RAG vs. Fine-tuning

Feature Comparison	RAG	Fine-Tuning
Knowledge Updates	Directly updating the retrieval knowledge base ensures that the information remains current without the need for frequent retraining, making it well-suited for dynamic data environments.	Stores static data, requiring retraining for knowledge and data updates.
External Knowledge	Proficient in leveraging external resources, particularly suitable for accessing documents or other structured/unstructured databases.	Can be utilized to align the externally acquired knowledge from pretraining with large language models, but may be less practical for frequently changing data sources.
Data Processing	Involves minimal data processing and handling.	Depends on the creation of high-quality datasets, and limited datasets may not result in significant performance improvements.
Model Customization	Focuses on information retrieval and integrating external knowledge but may not fully customize model behavior or writing style.	Allows adjustments of LLM behavior, writing style, or specific domain knowledge based on specific tones or terms.

RAG vs. Fine-tuning

Feature Comparison	RAG	Fine-Tuning
Interpretability	Responses can be traced back to specific data sources , providing higher interpretability and traceability.	Similar to a black box , it is not always clear why the model reacts a certain way, resulting in relatively lower interpretability.
Computational Resources	Depends on computational resources to support retrieval strategies and technologies related to databases . Additionally, it requires the maintenance of external data source integration and updates.	The preparation and curation of high-quality training datasets, defining fine-tuning objectives, and providing corresponding computational resources are necessary.
Latency Requirements	Involves data retrieval, which may lead to higher latency .	LLM after fine-tuning can respond without retrieval, resulting in lower latency .
Reducing Hallucinations	Inherently less prone to hallucinations as each answer is grounded in retrieved evidence.	Can help reduce hallucinations by training the model based on specific domain data but may still exhibit hallucinations when faced with unfamiliar input.
Ethical and Privacy Issues	Ethical and privacy concerns arise from the storage and retrieval of text from external databases .	Ethical and privacy concerns may arise due to sensitive content in the training data .

RAG Paradigms Shifting



Questions for Implementing RAG

- What and from where to retrieve?
 - Token, phrase, chunk, paragraph, entity, knowledge graph
- When to retrieve?
 - Single search, each token, every N tokens, adaptive search
- How to use the retrieved information?
 - Input/data layer, model/intermediate layer, output/prediction layer
- How to improve the retrieval for generation?
 - Prompting, query rewriting, finetuning, clarification

Techniques for Better RAG

- Enhanced embedding
 - Improve the embedding quality: chunk optimization and meta-data enrichment.
- Retrieval source optimization
 - Improve the retrieval by changing what to retrieve: phrase, prompt, cross-linguistic, LLM-generated text and LLM memory.
- Query optimization
 - Improve the query for the retrieval: rewriting and clarification.
- Embedding optimization
 - Train a domain-specific LLM: full retriever or adapter fine-tuning.
 - Hard to obtain the training data.
- Retrieval process optimization
 - Implement multi-turn generation process: iterative or adaptive generation processes.

Rewrite–Retrieve–Read

- Rewrite the query for better retrieval.
 - LLM rewriter with prompting
 - Trained rewriter by

Model	EM	F ₁
<i>HotpotQA</i>		
Direct	32.36	43.05
Retrieve-then-read	30.47	41.34
LLM rewriter	32.80	43.85
Trainable rewriter	34.38	45.97
<i>AmbigNQ</i>		
Direct	42.10	53.05
Retrieve-then-read	45.80	58.50
LLM rewriter	46.40	58.74
Trainable rewriter	47.80	60.71
<i>PopQA</i>		
Direct	41.94	44.61
Retrieve-then-read	43.20	47.53
LLM rewriter	46.00	49.74
Trainable rewriter	45.72	49.51

